M08 — Publicación mediante API Gateway con YARP · Teoría

Autor del curso: JAIME GALLO

Cómo estudiar el módulo

El módulo conserva los cuatro puntos y los cuarenta bloques del temario. Cada ejemplo se explica línea por línea, contando también las vacías; cada bloque añade una aplicación técnica a AceriaApp con fundamento, caso, límites y comprobación. Doce diagramas, tres por punto, se ofrecen en SVG y PNG. Las prácticas separadas contienen los archivos completos y las instrucciones de reconstrucción.

Punto de partida. El acumulado de M07 termina en 7.4, con 34 proyectos, 22 proyectos de pruebas, 403 pruebas y 42 documentos. Se conservan .NET 8, EF Core 8, SQL Server Express/LocalDB, Producción, Inventario, RabbitMQ y outbox/inbox. El gateway no modifica la lectura informativa del stock: no reserva ni descuenta existencias.

Recorrido. 8.1 modela y valida el contrato conceptual. 8.2 añade AceriaApp.Gateway con Yarp.ReverseProxy 2.2.0 sobre net8.0. 8.3 estudia selección y transformaciones con un alias limitado de Inventario. 8.4 reparte entre dos instancias con RoundRobin y salud activa. Los catálogos originales se conservan como ejercicios de análisis y se acompañan de comportamiento HTTP real.

Contrato del laboratorio. Gateway escucha en 5100; Producción en 5105; Inventario en 5106 y su segunda instancia final en 5107. Los prefijos públicos de Producción son /api/v1/ordenes, /api/v1/produccion y /api/v1/comunicacion; los de Inventario son /api/v1/materiales y /api/v1/planificaciones. El alias /inventario se limita a esos recursos de Inventario y elimina ese prefijo antes del envío. No publica diagnósticos internos mediante un comodín global. En Inventario se admiten GET y HEAD; las escrituras a sus recursos públicos se rechazan con 405.

Convenciones. Los fragmentos son ejemplos independientes y algunos reducen la configuración para estudiar una sola relación. Los nombres aceria-ordenes, aceria-inventario y aceria-clientes de un ejemplo de red son ASCII y conceptuales; no acreditan contenedores instalados ni un servicio Clientes ejecutable. Las direcciones y opciones completas del recorrido local se fijan en las prácticas. Los textos de catálogo que declaran autenticación, cuotas, ponderación o health checks no implementan esas capacidades.

Alcance y evidencia. El proxy conserva método, cuerpo, query y resultados de la API; no reintenta automáticamente escrituras ni inventa éxitos ante fallo remoto. Se distinguen pruebas del catálogo, pruebas con Kestrel controlado y recorridos entre servicios con SQL Server y RabbitMQ reales. Autenticación, rate limiting de producción, autoscaling y alta disponibilidad global quedan como capacidades de diseño, sin presentarlas como implementadas por un indicador booleano. Las cifras finales se obtienen de ejecuciones sobre la revisión concreta.

Punto 8.1 — Introducción al patrón API Gateway

Módulo 8: Publicación mediante API Gateway

Objetivos de aprendizaje:

- Comprender qué es un API Gateway y su papel en una arquitectura de microservicios.
- Identificar las responsabilidades del API Gateway y las que no le corresponden.
- Comprender la diferencia entre API Gateway y balanceador de carga.
- Comprender la relación entre API Gateway y los servicios de la acería.
- Añadir al proyecto `AceriaApp` las clases de validación de API Gateway y pruebas que verifiquen la validación, distinguiéndolas del comportamiento funcional del proxy.

Desarrollo teórico

Bloque 1 — Definición de API Gateway

Un API Gateway es un componente que actúa como punto de entrada único a un conjunto de servicios. El gateway recibe las peticiones dirigidas a su contrato público y las enruta al servicio correspondiente. El gateway oculta la topología interna de los servicios. Los clientes solo conocen la dirección del gateway. El gateway centraliza las preocupaciones transversales: autenticación, autorización, limitación de tasa, registro y monitorización. El gateway es la pieza que permite que los servicios internos sean autónomos sin exponer su ubicación.

```
API Gateway:

Cliente → Gateway → Servicio A.
                  → Servicio B.
                  → Servicio C.
```

- **Línea 1:** `API Gateway:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Cliente → Gateway → Servicio A.` → asocia Cliente con Gateway → Servicio A, según el alcance conceptual del ejemplo.
- **Línea 4:** `→ Servicio B.` → el gateway enruta al servicio B.
- **Línea 5:** `→ Servicio C.` → el gateway enruta al servicio C.

El API Gateway se caracteriza por tres propiedades. La primera es que es el punto de entrada único. La segunda es que oculta la topología interna. La tercera es que centraliza las preocupaciones transversales. Las tres propiedades permiten que los servicios internos sean autónomos.

La aplicación del API Gateway tiene dos consecuencias prácticas. La primera es que los clientes se simplifican: solo conocen un punto de entrada. La segunda es que se evita anunciar las direcciones internas como contrato público; impedir acceso directo requiere controles de red adicionales. La combinación de ambas permite que el sistema sea más seguro y más mantenible.

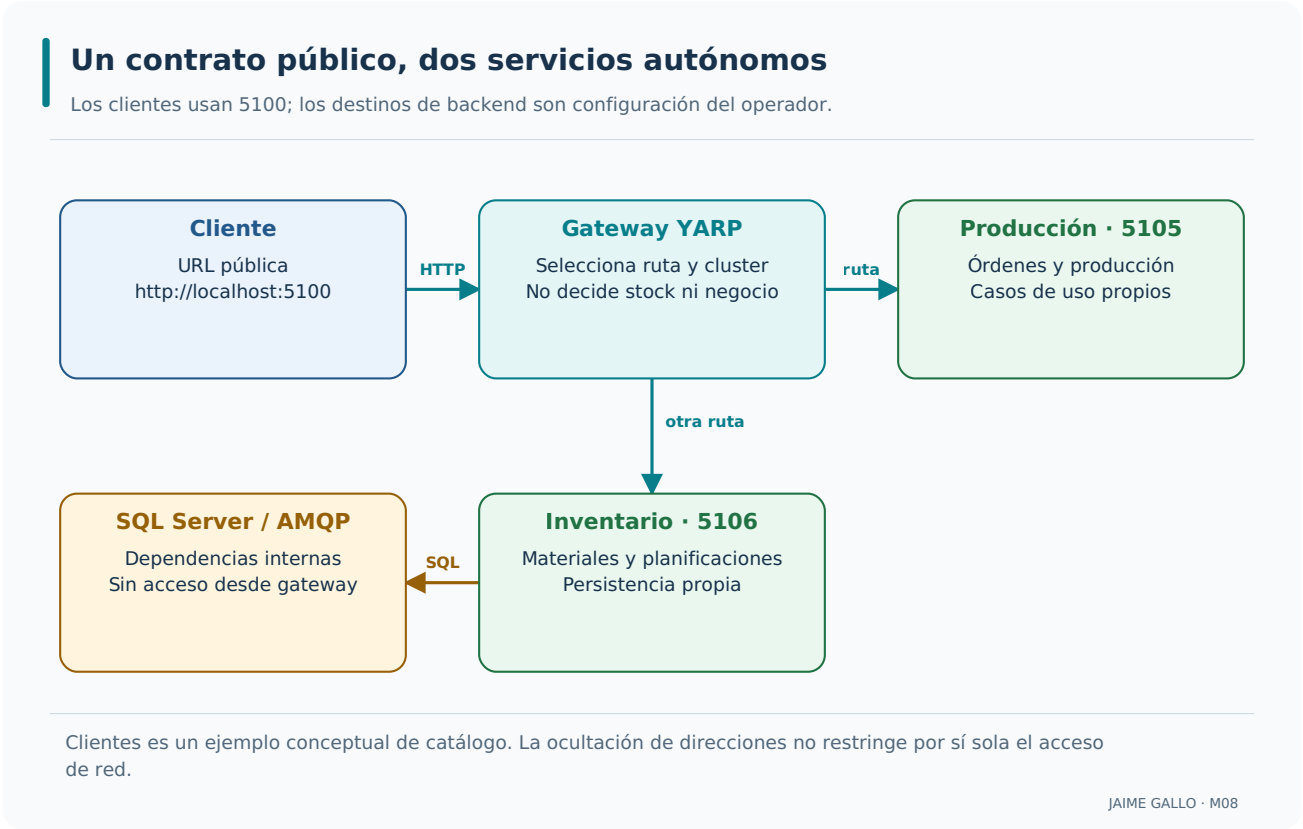
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La dirección pública es un contrato que puede mantenerse aunque cambien los procesos internos. El gateway es un proxy inverso: recibe HTTP como servidor y lo envía como cliente al destino configurado.

Ejemplo razonado. En AceriaApp el alumno usa localhost:5100. La ruta de órdenes conduce a Producción en 5105; materiales y planificaciones conducen a Inventario en 5106. El servicio responsable conserva sus casos de uso y su persistencia.

Riesgo, alternativa y límite. Ocultar la dirección no impide alcanzar un puerto conocido. El aislamiento exige reglas de red, exposición de puertos y políticas de acceso. El laboratorio mantiene los puertos internos accesibles para poder compararlos durante las pruebas.

Comprobación y aprendizaje. Compara la misma consulta por el gateway y por el servicio. Deben coincidir estado y significado del cuerpo; los registros deben permitir identificar ambos procesos. Detener el gateway debe afectar al acceso público sin detener los backends.



Interpretación del gráfico: El cliente conoce el gateway y cada ruta selecciona el servicio responsable. Producción e Inventario mantienen sus dependencias detrás de sus propios límites. La conexión interna de Inventario con SQL/AMQP no implica acceso del gateway a esos recursos. Clientes no aparece como proceso porque pertenece al catálogo conceptual. Los puertos internos siguen disponibles para comprobar el laboratorio; la figura no acredita aislamiento de red.

Bloque 2 — Responsabilidades del API Gateway

El patrón API Gateway puede asumir cuatro tipos de responsabilidad según las necesidades del sistema. La primera es el enrutamiento: dirigir cada petición al servicio correspondiente según la ruta. La segunda es la traducción: convertir el protocolo externo en el protocolo interno. La tercera es la agregación: combinar respuestas de varios servicios en una única respuesta. La cuarta es la aplicación de políticas transversales: autenticación, autorización, limitación de tasa, registro y monitorización. Cada responsabilidad se implementa de forma independiente.

```
Responsabilidades del API Gateway:

Enrutamiento        → dirigir la petición al servicio.
Traducción          → convertir protocolos.
Agregación          → combinar respuestas.
Políticas transversales → autenticación, autorización, rate limiting.
```

- **Línea 1:** `Responsabilidades del API Gateway:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Enrutamiento → dirigir la petición al servicio.` → asocia Enrutamiento con dirigir la petición al servicio, según el alcance conceptual del ejemplo.
- **Línea 4:** `Traducción → convertir protocolos.` → asocia Traducción con convertir protocolos, según el alcance conceptual del ejemplo.
- **Línea 5:** `Agregación → combinar respuestas.` → asocia Agregación con combinar respuestas, según el alcance conceptual del ejemplo.
- **Línea 6:** `Políticas transversales → autenticación, autorización, rate limiting.` → asocia Políticas transversales con autenticación, autorización, rate limiting, según el alcance conceptual del ejemplo.

Las responsabilidades se caracterizan por tres propiedades. La primera es que cada responsabilidad es independiente. La segunda es que combinan configuración declarativa y componentes específicos cuando la función lo necesita. La tercera es que el alcance depende de las políticas y rutas configuradas. Las tres propiedades permiten que el gateway sea configurable y mantenible.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Enrutamiento es la responsabilidad mínima. Traducción de protocolos y agregación son posibilidades arquitectónicas que requieren código y decisiones adicionales; no aparecen automáticamente al registrar un proxy inverso.

Ejemplo razonado. El gateway docente conduce órdenes a Producción y materiales a Inventario. No compone una respuesta de cliente con varios servicios ni consume mensajes RabbitMQ para responder HTTP. Su configuración determina la superficie publicada.

Riesgo, alternativa y límite. Agregar respuestas puede acoplar latencias y fallos: una petición pasa a depender de varios servicios. Antes de introducir esa función, define qué ocurre ante datos parciales y qué parte pertenece a una aplicación específica, por ejemplo un BFF.

Comprobación y aprendizaje. Localiza una ruta pública, su ClusterId y sus destinos. Esa cadena explica quién recibe una petición. Una prueba que solo comprueba nombres de servicios valida la declaración; una petición real acredita el enrutamiento efectivo.

Bloque 3 — Lo que un API Gateway no debe hacer

El API Gateway no debe contener reglas de negocio. Las reglas de negocio residen en los servicios internos. El gateway no debe transformar el modelo del dominio: solo enruta y aplica políticas. El gateway no debe ser un punto único de fallo: se despliega con redundancia. El gateway no debe conservar estado de negocio o de sesión exclusivo de una instancia; puede mantener estado operativo como conexiones y salud de destinos. El gateway no debe acceder directamente a la base de datos de los servicios.

```
Lo que un API Gateway no debe hacer:

- Contener reglas de negocio.
- Transformar el modelo del dominio.
- Ser un punto único de fallo.
- Almacenar estado de negocio o de sesión exclusivo de una instancia.
- Acceder directamente a la base de datos.
```

- **Línea 1:** `Lo que un API Gateway no debe hacer:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `- Contener reglas de negocio.` → las reglas residen en los servicios.
- **Línea 4:** `- Transformar el modelo del dominio.` → solo enruta.
- **Línea 5:** `- Ser un punto único de fallo.` → se despliega con redundancia.
- **Línea 6:** `- Almacenar estado de negocio o de sesión exclusivo de una instancia.` → evita que una sesión o un dato de negocio dependa de una única copia local; las conexiones y la salud son estado técnico permitido.
- **Línea 7:** `- Acceder directamente a la base de datos.` → solo enruta.

La delimitación de responsabilidades tiene dos consecuencias prácticas. La primera es que el gateway se mantiene simple. La segunda es que los servicios internos conservan su autonomía. La combinación de ambas permite que el gateway sea sustituible sin afectar a la lógica de negocio.

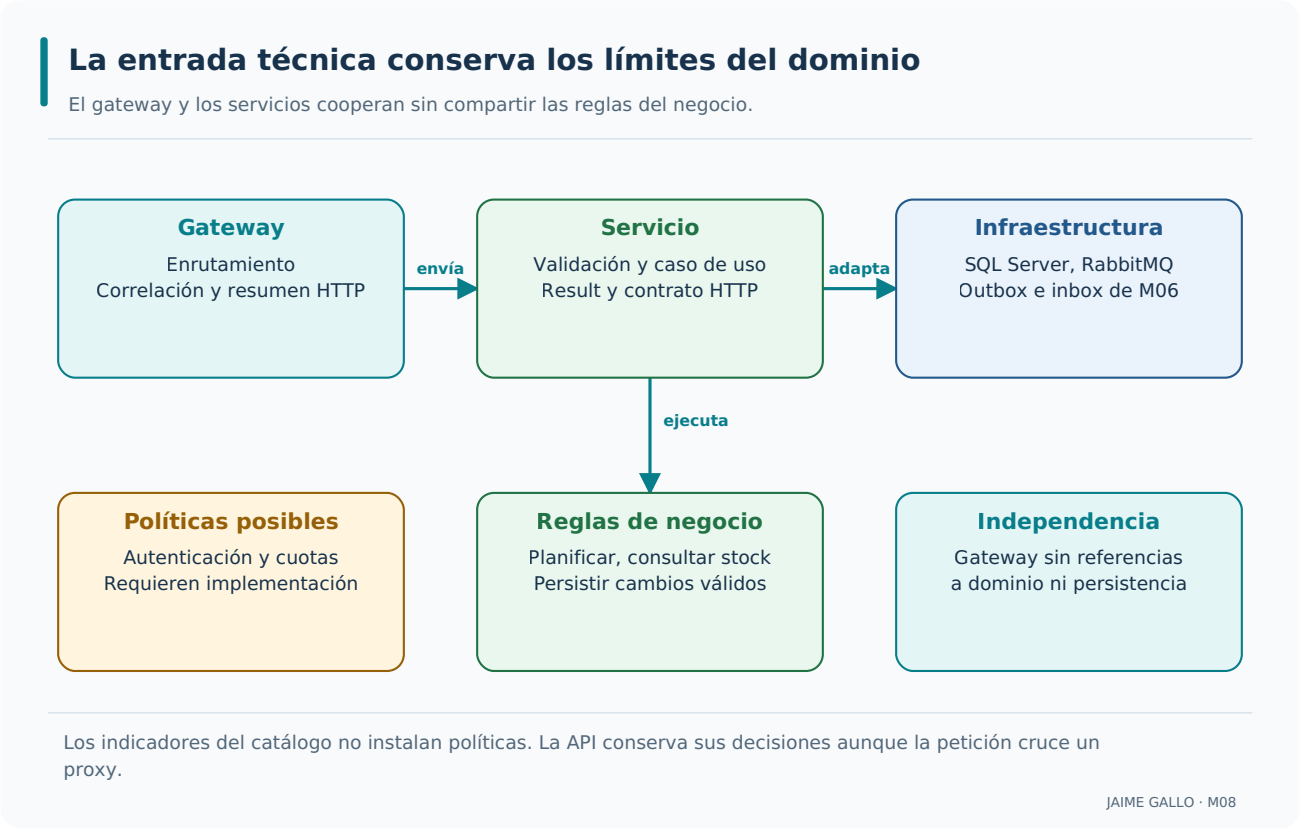
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El límite hexagonal coloca las decisiones del negocio en los casos de uso y el dominio. El gateway es un adaptador de entrada independiente; no necesita referencias a entidades, repositorios ni DbContext para reenviar HTTP.

Ejemplo razonado. La regla que rechaza una planificación no válida sigue ejecutándose en Producción. Un cambio de ruta pública no modifica esa regla. SQL Server y RabbitMQ permanecen detrás de sus servicios, fuera del proceso AceriaApp.Gateway.

Riesgo, alternativa y límite. Sin estado de sesión exclusivo no significa sin ningún estado: YARP mantiene conexiones, configuración y estados operativos de salud. Esos datos técnicos no deben convertirse en la única copia de información funcional necesaria para atender una petición.

Comprobación y aprendizaje. Revisa las referencias del proyecto Gateway y el camino de una escritura. Debe ser el servicio quien valide y persista. Comprueba también que cada instancia del gateway podría arrancar con la misma configuración sin importar una sesión local de negocio.



Interpretación del gráfico: La fila superior identifica el recorrido técnico hasta la infraestructura de cada servicio. Las tarjetas inferiores distinguen políticas que podrían añadirse, reglas que sí pertenecen a los casos de uso y referencias que el gateway debe evitar. La ausencia de un enlace a las políticas posibles es deliberada: no se presentan autenticación y cuotas como componentes implementados. Una nueva regla de planificación se modifica en su servicio, no en el proxy.

Bloque 4 — Diferencia entre API Gateway y balanceador

El API Gateway y el balanceador de carga tienen responsabilidades distintas. El balanceador distribuye el tráfico entre varias instancias del mismo servicio. El gateway enruta el tráfico entre servicios distintos. Un balanceador puede trabajar en capa de transporte o en capa de aplicación. El gateway trabaja en la capa de aplicación. El balanceador no conoce la lógica de los servicios. El gateway conoce las rutas y aplica políticas. La combinación de ambos permite que el sistema sea escalable y seguro.

```
Diferencia entre API Gateway y balanceador:

Balanceador → distribuye entre instancias del mismo servicio.
Gateway     → enruta entre servicios distintos.
```

- **Línea 1:** `Diferencia entre API Gateway y balanceador:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Balanceador → distribuye entre instancias del mismo servicio.` → asocia Balanceador con distribuye entre instancias del mismo servicio, según el alcance conceptual del ejemplo.
- **Línea 4:** `Gateway → enruta entre servicios distintos.` → asocia Gateway con enruta entre servicios distintos, según el alcance conceptual del ejemplo.

La diferencia se caracteriza por tres propiedades. La primera es que el balanceador distribuye entre instancias y puede operar en capa de transporte o de aplicación. La segunda es que el gateway

trabaja en capa de aplicación. La tercera es que ambos se complementan. Las tres propiedades permiten que el sistema sea escalable y seguro.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Gateway y balanceador describen funciones distintas que pueden convivir en un producto. Existen balanceadores de capa 4 y de capa 7. YARP opera sobre HTTP y también puede seleccionar una instancia dentro de un cluster.

Ejemplo razonado. Primero, /api/v1/materiales selecciona el servicio Inventario. Después, en 8.4, RoundRobin elige entre sus procesos 5106 y 5107. La primera decisión se basa en la ruta; la segunda se basa en el conjunto de destinos disponibles.

Riesgo, alternativa y límite. Añadir réplicas del gateway protege el punto de entrada; añadir réplicas de Inventario aumenta sus destinos. Son escalados diferentes. Ninguno elimina por sí solo un fallo compartido de SQL Server ni resuelve una configuración errónea distribuida a todas las réplicas.

Comprobación y aprendizaje. Dibuja ambas decisiones para una consulta. Si la ruta cambia de órdenes a materiales, cambia el servicio; si cambia solo la identidad de instancia de Inventario, ha actuado el balanceo conservando el mismo contrato HTTP.

Bloque 5 — Patrones de API Gateway

El patrón API Gateway admite varias variantes. La primera es el gateway único: un solo gateway para todos los servicios. La segunda es el gateway por cliente: un gateway para cada tipo de cliente. La tercera es el gateway por dominio: un gateway para cada dominio funcional. La cuarta es el BFF (Backend For Frontend): un gateway específico para cada interfaz de usuario. La elección depende del número de clientes y de la diversidad de sus necesidades.

```
Patrones de API Gateway:

Gateway único       → un gateway para todos.
Gateway por cliente → un gateway por cliente.
Gateway por dominio → un gateway por dominio.
BFF                 → un gateway por interfaz.
```

- **Línea 1:** `Patrones de API Gateway:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Gateway único → un gateway para todos.` → asocia Gateway único con un gateway para todos, según el alcance conceptual del ejemplo.
- **Línea 4:** `Gateway por cliente → un gateway por cliente.` → asocia Gateway por cliente con un gateway por cliente, según el alcance conceptual del ejemplo.
- **Línea 5:** `Gateway por dominio → un gateway por dominio.` → asocia Gateway por dominio con un gateway por dominio, según el alcance conceptual del ejemplo.
- **Línea 6:** `BFF → un gateway por interfaz.` → asocia BFF con un gateway por interfaz, según el alcance conceptual del ejemplo.

Los patrones se caracterizan por tres propiedades. La primera es que cada patrón tiene un propósito específico. La segunda es que la elección depende del contexto. La tercera es que los patrones se pueden combinar. Las tres propiedades permiten que el gateway se adapte a las necesidades del sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un BFF adapta una API a las necesidades de una interfaz concreta. Un gateway común prioriza contratos y políticas compartidas. La separación por dominio puede reflejar propiedad de equipos o distintos ciclos de publicación.

Ejemplo razonado. AceriaApp usa un único punto de entrada docente para que el recorrido sea observable. Los catálogos contemplan órdenes, inventario y clientes; Clientes permanece como ejemplo conceptual, sin introducir un tercer backend que no existe en el proyecto acumulativo.

Riesgo, alternativa y límite. Crear un gateway por cliente sin diferencias funcionales multiplica despliegues y configuración. A la inversa, concentrar toda composición de pantallas en un gateway común puede convertirlo en una aplicación con demasiados cambios y dependencias.

Comprobación y aprendizaje. Para justificar otra variante, escribe qué necesidad cambia: datos, autorización, latencia o versión. Si la única diferencia es la dirección del backend, una configuración por entorno suele resolverla sin añadir otro patrón.

Bloque 6 — Funciones transversales del API Gateway

El API Gateway puede centralizar cinco funciones transversales principales, si se registran sus componentes y políticas. La primera es la autenticación: verificar la identidad del cliente. La segunda es la autorización: verificar que el cliente tiene permisos. La tercera es la limitación de tasa: evitar el abuso del servicio. La cuarta es el registro: registrar todas las peticiones. La quinta es la monitorización: medir el rendimiento y la disponibilidad. Su aplicación combina configuración, registro de componentes y código cuando la política lo requiere.

```
Funciones transversales:

Autenticación       → verificar identidad.
Autorización        → verificar permisos.
Limitación de tasa  → evitar abuso.
Registro            → registrar peticiones.
Monitorización      → medir rendimiento.
```

- **Línea 1:** `Funciones transversales:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Autenticación → verificar identidad.` → asocia Autenticación con verificar identidad, según el alcance conceptual del ejemplo.
- **Línea 4:** `Autorización → verificar permisos.` → asocia Autorización con verificar permisos, según el alcance conceptual del ejemplo.
- **Línea 5:** `Limitación de tasa → evitar abuso.` → asocia Limitación de tasa con evitar abuso, según el alcance conceptual del ejemplo.
- **Línea 6:** `Registro → registrar peticiones.` → asocia Registro con registrar peticiones, según el alcance conceptual del ejemplo.
- **Línea 7:** `Monitorización → medir rendimiento.` → asocia Monitorización con medir rendimiento, según el alcance conceptual del ejemplo.

Las funciones se caracterizan por tres propiedades. La primera es que su alcance se define por las políticas y el pipeline. La segunda es que se configuran de forma declarativa. La tercera es que se pueden desactivar por ruta. Las tres propiedades permiten que el gateway sea configurable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Una preocupación transversal requiere registro, configuración y orden de ejecución. Tener una propiedad booleana llamada AplicaAutenticacion no ejecuta un autenticador; una cuota declarada necesita un limitador y una política aplicable a la ruta.

Ejemplo razonado. El catálogo original conserva los indicadores para expresar una revisión de diseño. La ampliación funcional implementa correlación, resumen HTTP seguro y encaminamiento; no presenta esos indicadores como acreditación de autenticación, autorización o cuotas de producción.

Riesgo, alternativa y límite. El registro indiscriminado de peticiones puede copiar cuerpos y credenciales. Se conserva el criterio de M07: propiedades técnicas seleccionadas y mensajes públicos seguros. La observación debe aportar diagnóstico sin introducir datos sensibles innecesarios.

Comprobación y aprendizaje. Distingue en una revisión tres evidencias: intención del catálogo, registro de middleware y comportamiento observado. Para afirmar que una política opera, una prueba debe mostrar tanto una petición admitida como otra rechazada por esa política.

Bloque 7 — API Gateway y seguridad

El API Gateway es el punto donde se aplican las políticas de seguridad. La seguridad incluye la autenticación, la autorización, la validación de la entrada y la protección contra ataques. El gateway puede validar la petición antes de enrutarla. El gateway puede rechazar las peticiones mal formadas. El gateway puede aplicar cuotas por cliente. La seguridad en el gateway es la primera línea de defensa del sistema.

```
Seguridad en el API Gateway:

Autenticación       → verificar identidad.
Autorización        → verificar permisos.
Validación entrada  → comprobar formato.
Protección ataques  → limitar tasa ayuda a mitigar abuso.
```

- **Línea 1:** `Seguridad en el API Gateway:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Autenticación → verificar identidad.` → asocia Autenticación con verificar identidad, según el alcance conceptual del ejemplo.
- **Línea 4:** `Autorización → verificar permisos.` → asocia Autorización con verificar permisos, según el alcance conceptual del ejemplo.
- **Línea 5:** `Validación entrada → comprobar formato.` → asocia Validación entrada con comprobar formato, según el alcance conceptual del ejemplo.
- **Línea 6:** `Protección ataques → limitar tasa ayuda a mitigar abuso.` → la limitación de tasa ayuda a mitigar abuso, sin constituir protección completa contra todas las clases de ataque.

La seguridad se caracteriza por tres propiedades. La primera es que se aplica en el punto de entrada. La segunda es que centraliza las políticas. La tercera es que protege a los servicios internos. Estas propiedades contribuyen a un diseño de seguridad que también necesita controles en servicios y red.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El borde es un lugar adecuado para aplicar controles compartidos, pero cada servicio conserva la autorización de sus operaciones. La validación técnica en el proxy no reemplaza las invariantes del dominio ni la validación del caso de uso.

Ejemplo razonado. El laboratorio delimita los prefijos publicados y valida la correlación de entrada. Un valor inválido se sustituye en el borde antes de propagarse. No se identifica al usuario mediante X-Canal, que es una marca técnica estable del recorrido.

Riesgo, alternativa y límite. Si un backend sigue accesible, un cliente puede intentar evitar el gateway. Además, confiar en cabeceras reenviadas exige conocer los proxies autorizados. Una cabecera que contiene un nombre de usuario no prueba identidad por sí misma.

Comprobación y aprendizaje. Intenta una ruta no publicada y un endpoint interno de diagnóstico. Verifica que no se alcanzan mediante un comodín abierto. Separa ese ensayo de una revisión de autenticación, que requiere un mecanismo y credenciales concretos fuera de este alcance.

Bloque 8 — API Gateway y observabilidad

El API Gateway es el punto donde se concentra la observabilidad. El gateway registra todas las peticiones. El gateway mide la latencia de cada petición. El gateway puede contar peticiones por servicio y código; el laboratorio aporta resúmenes HTTP, sin desplegar un backend de métricas. El gateway propaga el identificador de correlación. La observabilidad en el gateway es la base del diagnóstico del sistema. La observabilidad se integra con las herramientas de monitorización.

```
Observabilidad en el API Gateway:

Registro          → registrar todas las peticiones.
Latencia          → medir el tiempo de cada petición.
Contadores        → contar peticiones por servicio, capacidad posible.
Correlación       → propagar identificador.
```

- **Línea 1:** `Observabilidad en el API Gateway:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Registro → registrar todas las peticiones.` → asocia Registro con registrar todas las peticiones, según el alcance conceptual del ejemplo.
- **Línea 4:** `Latencia → medir el tiempo de cada petición.` → asocia Latencia con medir el tiempo de cada petición, según el alcance conceptual del ejemplo.
- **Línea 5:** `Contadores → contar peticiones por servicio, capacidad posible.` → muestra una capacidad posible de observación; el laboratorio registra resúmenes HTTP sin desplegar un backend de métricas.
- **Línea 6:** `Correlación → propagar identificador.` → asocia Correlación con propagar identificador, según el alcance conceptual del ejemplo.

La observabilidad se caracteriza por tres propiedades. La primera es que se concentra en el gateway. La segunda es que se integra con las herramientas existentes. La tercera es que contribuye al diagnóstico entre procesos cuando se integra con las señales de los servicios. Las tres propiedades permiten que el sistema sea observable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La latencia del gateway mide el recorrido que él observa; no explica por sí sola cuánto tiempo consumió cada dependencia. La correlación une eventos, mientras una traza distribuida completa añade relaciones temporales entre operaciones.

Ejemplo razonado. Una consulta de Producción que llama a Inventario puede compartir CorrelationId con el resumen HTTP del gateway. El nombre del servicio y la identidad de instancia permiten ubicar el emisor sin confundir varios procesos que escriben JSON simultáneamente.

Riesgo, alternativa y límite. Registrar la query o el cuerpo completo para diagnosticar transformaciones puede exponer información. Las pruebas controladas pueden inspeccionar esos valores en un servidor de prueba sin habilitar ese volcado en los logs normales del laboratorio.

Comprobación y aprendizaje. Envía una correlación válida y otra inválida. La primera se conserva; la segunda debe generar un valor efectivo coherente entre respuesta y servicios. Compara una consulta correcta con una caída de backend sin interpretar todo error como fallo de negocio.



Interpretación del gráfico: La fila superior sigue la petición y la inferior el regreso de la respuesta. El estado funcional lo decide la API y el gateway conserva su significado. Location necesita una dirección que el cliente pueda seguir por el contrato público. El identificador efectivo de correlación permite relacionar los dos sentidos. Un fallo entre procesos produce un error técnico; el borde no inventa un cuerpo de éxito para ocultarlo.

Bloque 9 — Ventajas e inconvenientes del API Gateway

El API Gateway aporta cuatro ventajas principales. La primera es la simplificación del cliente: el cliente solo conoce un punto de entrada. La segunda es la centralización de políticas: las políticas se aplican en un único punto. La tercera es la ocultación de la topología en el contrato público, complementada por controles de exposición de red. La cuarta es la observabilidad: el gateway concentra el registro y la monitorización. El API Gateway introduce tres inconvenientes. El primero es el punto único de fallo: si el gateway falla, todo el sistema falla. El segundo es la latencia adicional: cada petición pasa por el gateway. El tercero es la complejidad operativa: el gateway requiere configuración y monitorización.

```
Ventajas:

Simplificación del cliente.
Centralización de políticas.
Ocultación de la topología.
Observabilidad.

Inconvenientes:

Punto único de fallo.
Latencia adicional.
Complejidad operativa.
```

- **Línea 1:** `Ventajas:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Simplificación del cliente.` → primera ventaja.
- **Línea 4:** `Centralización de políticas.` → segunda ventaja.
- **Línea 5:** `Ocultación de la topología.` → tercera ventaja.
- **Línea 6:** `Observabilidad.` → cuarta ventaja.
- **Línea 7:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 8:** `Inconvenientes:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 9:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 10:** `Punto único de fallo.` → primer inconveniente.
- **Línea 11:** `Latencia adicional.` → segundo inconveniente.
- **Línea 12:** `Complejidad operativa.` → tercer inconveniente.

Las ventajas e inconvenientes se caracterizan por tres propiedades. La primera es que las ventajas se materializan en sistemas con varios servicios. La segunda es que los inconvenientes se mitigan con redundancia y monitorización. La tercera es que la decisión se documenta en un ADR. Las tres propiedades permiten que la adopción del gateway sea razonada.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La adopción añade una dependencia en el camino público. Su beneficio debe evaluarse junto con latencia, coste operativo, disponibilidad y propiedad de la configuración. La redundancia es una estrategia de despliegue, no una propiedad automática de la biblioteca.

Ejemplo razonado. En el curso un solo proceso facilita el seguimiento del tráfico. Dos instancias de Inventario permiten estudiar reparto y recuperación, pero el gateway local y la base compartida continúan siendo componentes cuyo fallo afecta al recorrido.

Riesgo, alternativa y límite. Más reglas en el borde aumentan las combinaciones que comprobar. Una actualización que publica accidentalmente un prefijo interno puede afectar a todos los servicios. Conviene revisar la superficie pública como contrato y mantener una alternativa de reversión.

Comprobación y aprendizaje. El ADR-031 explica la decisión y sus consecuencias. Contrasta una ventaja concreta, como no cambiar la URL del cliente al mover Inventario, con su coste verificable: otro proceso que arrancar, configurar, observar y probar.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica el patrón API Gateway en el módulo 8. El gateway se implementa con YARP. El catálogo estudia órdenes, inventario y clientes; el gateway ejecutable publica recursos de Producción e Inventario. Clientes es un ejemplo conceptual. El gateway docente aplica registro y correlación; la limitación de tasa se conserva como política conceptual, sin presentarla como implementada. El gateway expone un único punto de entrada. La validación del gateway se realiza con un test que verifica la coherencia de las rutas.

```
Aplicación al proyecto del curso:

Gateway          → YARP.
Rutas            → catálogo conceptual; Producción e Inventario ejecutables.
Políticas        → registro y correlación; cuotas conceptuales.
Punto de entrada → único.
Validación       → test de coherencia.
```

- **Línea 1:** `Aplicación al proyecto del curso:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Gateway → YARP.` → asocia Gateway con YARP, según el alcance conceptual del ejemplo.
- **Línea 4:** `Rutas → catálogo conceptual; Producción e Inventario ejecutables.` → asocia Rutas con catálogo conceptual; Producción e Inventario ejecutables, según el alcance conceptual del ejemplo.
- **Línea 5:** `Políticas → registro y correlación; cuotas conceptuales.` → asocia Políticas con registro y correlación; cuotas conceptuales, según el alcance conceptual del ejemplo.
- **Línea 6:** `Punto de entrada → único.` → asocia Punto de entrada con único, según el alcance conceptual del ejemplo.
- **Línea 7:** `Validación → test de coherencia.` → asocia Validación con test de coherencia, según el alcance conceptual del ejemplo.

La aplicación del API Gateway prepara el proyecto para los puntos 8.2, 8.3 y 8.4. El punto 8.2 estudiará la configuración básica de YARP. El punto 8.3 estudiará el enrutamiento de peticiones. El punto 8.4 estudiará el balanceo de carga básico. El punto 8.1 establece la base que se aplicará en los tres puntos siguientes.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El punto 8.1 prepara un modelo explícito para revisar rutas sin confundirlo con el proxy. RutaGateway describe datos; ValidadorApiGateway comprueba decisiones; CatalogoRutas proporciona un conjunto reproducible para estudiar esas comprobaciones.

Ejemplo razonado. Se conservan las tres rutas conceptuales de órdenes, inventario y clientes con autenticación, tasa y correlación declaradas. La integración posterior publica recursos existentes de Producción e Inventario, y conserva Clientes únicamente como ejemplo de análisis.

Riesgo, alternativa y límite. Un test verde del catálogo no prueba que el puerto 5100 esté escuchando ni que la API de destino haya contestado. Tampoco prueba la aplicación efectiva de sus tres políticas. Los ensayos de proxy se incorporan a partir de la etapa 8.2.

Comprobación y aprendizaje. El reto calcula total de rutas, servicios distintos y rutas con autenticación declarada: el catálogo produce 3, 3 y 3. Explica el significado de esas cifras y después identifica qué evidencia adicional se necesita para comprobar tráfico real.

Resumen rápido de la teoría

- Un API Gateway es un componente que actúa como punto de entrada único a un conjunto de servicios.
- El gateway oculta la topología interna y centraliza las preocupaciones transversales.
- Las responsabilidades del gateway son enrutamiento, traducción, agregación y políticas transversales.
- El gateway mantiene fuera las reglas y persistencia de negocio; evita estado de sesión exclusivo y requiere redundancia para mitigar un fallo del punto de entrada.
- El balanceador distribuye entre instancias del mismo servicio; el gateway enruta entre servicios distintos.
- Los patrones de gateway son gateway único, por cliente, por dominio y BFF.
- Las funciones transversales son autenticación, autorización, limitación de tasa, registro y monitorización.
- El gateway es el punto donde se aplican las políticas de seguridad y se concentra la observabilidad.
- Las ventajas son simplificación, centralización, ocultación y observabilidad. Los inconvenientes son punto único de fallo, latencia adicional y complejidad operativa.

Punto 8.2 — Configuración básica de YARP

Módulo 8: Publicación mediante API Gateway

Objetivos de aprendizaje:

- Comprender qué es YARP y su papel como implementación del patrón API Gateway.
- Identificar los conceptos de ruta y cluster en YARP.
- Comprender la configuración de YARP en `appsettings.json`.
- Comprender la integración de YARP con ASP.NET Core.
- Añadir al proyecto `AceriaApp` las clases de validación de la configuración de YARP y pruebas que verifiquen la validación, distinguiéndolas del comportamiento funcional del proxy.

Desarrollo teórico

Bloque 1 — Definición de YARP

YARP es el acrónimo de Yet Another Reverse Proxy. Es una biblioteca de .NET para construir proxies inversos y gateways. YARP fue desarrollada por Microsoft y se distribuye como paquete NuGet. La biblioteca se integra con ASP.NET Core y aprovecha el modelo de middleware del framework. YARP permite enrutar peticiones HTTP a servicios internos según reglas configurables. La configuración se realiza en `appsettings.json` o en código. YARP es la implementación del patrón API Gateway que el curso usa.

```
YARP:

Cliente → YARP → Servicio interno.
```

- **Línea 1:** `YARP:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Cliente → YARP → Servicio interno.` → YARP actúa como intermediario.

YARP se caracteriza por tres propiedades. La primera es que es una biblioteca de .NET. La segunda es que se integra con ASP.NET Core. La tercera es que la configuración es declarativa. Las tres propiedades permiten que el gateway sea configurable y mantenible.

La aplicación de YARP tiene dos consecuencias prácticas. La primera es que el gateway se construye con el mismo stack que los servicios. La segunda es que la configuración se versiona con el código. La combinación de ambas permite que el gateway sea coherente con el resto del sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. YARP aporta componentes de proxy integrados en el modelo de ASP.NET Core. El proyecto que lo hospeda decide configuración, observación, políticas y despliegue. El paquete docente está fijado a Yarp.ReverseProxy 2.2.0 y el proyecto a net8.0.

Ejemplo razonado. AceriaApp.Gateway es un proceso separado de Producción e Inventario. Tener el mismo lenguaje no exige compartir el dominio: el contrato entre procesos sigue siendo HTTP, con sus códigos y cuerpos.

Riesgo, alternativa y límite. Actualizar una biblioteca puede cambiar contratos o valores predeterminados. Se estudian aquí las capacidades usadas por la versión fijada; los archivos completos de las prácticas son la referencia reproducible para paquetes y opciones de arranque.

Comprobación y aprendizaje. Identifica TargetFramework y PackageReference en el proyecto Gateway. Arranca el proceso y consulta una ruta publicada con backend disponible. La carga correcta de un paquete acredita instalación; la respuesta reenviada acredita ejecución del proxy.

Bloque 2 — Concepto de ruta en YARP

Una ruta en YARP define la relación entre una petición entrante y un cluster destino. La ruta tiene un identificador único, un patrón de coincidencia y un cluster asociado. El patrón de coincidencia se expresa con la sintaxis de rutas de ASP.NET Core. La ruta puede incluir transformaciones que modifican la petición antes de enviarla al destino. La ruta puede incluir políticas de autorización y de limitación de tasa. La ruta es la unidad de configuración del enrutamiento.

```
{
  "Routes": {
    "ordenes-route": {
      "ClusterId": "ordenes-cluster",
      "Match": {
        "Path": "/api/v1/ordenes/{**catch-all}"
      }
    }
  }
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Routes": {` → abre la sección de rutas.
- **Línea 3:** `"ordenes-route": {` → declara la ruta de órdenes.
- **Línea 4:** `"ClusterId": "ordenes-cluster",` → asocia la ruta al cluster.
- **Línea 5:** `"Match": {` → abre la sección de coincidencia.
- **Línea 6:** `"Path": "/api/v1/ordenes/{**catch-all}"` → patrón de coincidencia.
- **Línea 7:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 8:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 9:** `}` → cierra el objeto abierto en el contexto anterior.

- **Línea 10:** } → cierra el objeto abierto en el contexto anterior.

La ruta se caracteriza por tres propiedades. La primera es que tiene un identificador único. La segunda es que tiene un patrón de coincidencia. La tercera es que está asociada a un cluster. Las tres propiedades permiten que el enrutamiento sea declarativo.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Una ruta relaciona condiciones de entrada con un cluster. Match decide elegibilidad; ClusterId decide el conjunto de backend; Transforms adapta la petición seleccionada. Cada parte responde a una pregunta distinta y puede revisarse por separado.

Ejemplo razonado. Una regla sobre /api/v1/ordenes conduce a Producción. Las rutas de materiales y planificaciones conducen a Inventario. No se publica automáticamente toda ruta que un servicio pueda atender; cada prefijo público se elige deliberadamente.

Riesgo, alternativa y límite. Un catch-all global simplifica un ejemplo, pero puede publicar administración o diagnóstico del backend. Una lista de métodos también forma parte del contrato: omitir un método necesario puede impedir una operación válida y admitir todos puede ampliar la superficie.

Comprobación y aprendizaje. Ensaya una URL admitida, otra fuera de prefijo y un método no autorizado por la regla correspondiente. Comprueba en el servidor de prueba si recibió la petición; una respuesta de error aislada no identifica siempre quién la originó.

Bloque 3 — Concepto de cluster en YARP

Un cluster en YARP define el conjunto de destinos a los que se enruta una petición. El cluster tiene un identificador único y una lista de destinos. Cada destino tiene una dirección y puede incluir Health, Host y Metadata; una ponderación requiere una política personalizada, no una propiedad Weight incorporada. El cluster puede incluir una política de balanceo. El cluster puede incluir una política de salud. El cluster es la unidad de configuración del destino.

```
{
  "Clusters": {
    "ordenes-cluster": {
      "Destinations": {
        "ordenes-primary": {
          "Address": "http://aceria-ordenes:5001/"
        }
      }
    }
  }
}
```

- **Línea 1:** { → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Clusters": {` → abre la sección de clusters.
- **Línea 3:** `"ordenes-cluster": {` → declara el cluster de órdenes.
- **Línea 4:** `"Destinations": {` → abre la sección de destinos.
- **Línea 5:** `"ordenes-primary": {` → declara el destino principal.
- **Línea 6:** `"Address": "http://aceria-ordenes:5001/"` → declara una dirección de ejemplo con nombre ASCII; debe resolverse en la red del emisor.
- **Línea 7:** } → cierra el objeto abierto en el contexto anterior.
- **Línea 8:** } → cierra el objeto abierto en el contexto anterior.
- **Línea 9:** } → cierra el objeto abierto en el contexto anterior.

- **Línea 10:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 11:** `}` → cierra el objeto abierto en el contexto anterior.

El cluster se caracteriza por tres propiedades. La primera es que tiene un identificador único. La segunda es que contiene una lista de destinos. La tercera es que puede incluir políticas. Las tres propiedades permiten que el destino sea configurable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El cluster agrupa destinos intercambiables para el contrato de una ruta. Las políticas de balanceo, transporte y salud pertenecen a ese conjunto. El identificador de destino es una clave de configuración y no un dato que el cliente elige libremente.

Ejemplo razonado. Inventario comienza con una dirección local y termina con dos, 5106 y 5107. Ambas deben atender materiales y planificaciones con semántica compatible. Un destino de Producción no sería una réplica válida para ese cluster.

Riesgo, alternativa y límite. DestinationConfig expone Address, Health, Host y Metadata; no incluye Weight. Se puede conservar una intención de ponderación en un modelo docente, pero aplicarla requiere una política personalizada que interprete los metadatos.

Comprobación y aprendizaje. Revisa que cada URI incluya esquema y puerto correctos y que el backend escuche allí. El indicador TieneDireccionValida del catálogo solo expresa una declaración; la validación funcional requiere usar direcciones concretas y comprobar una respuesta.

Bloque 4 — Relación entre ruta y cluster

La relación entre ruta y cluster es de muchos a uno. Varias rutas pueden apuntar al mismo cluster. Un cluster puede recibir tráfico de varias rutas. La separación entre ruta y cluster permite que el enrutamiento y el destino evolucionen de forma independiente. La ruta define qué peticiones se enrutan. El cluster define a dónde se enrutan. La separación es la base de la flexibilidad de YARP.

```
Relación ruta-cluster:

Ruta A referencia Cluster X.
Ruta B referencia Cluster X.
Ruta C referencia Cluster Y.
```

- **Línea 1:** `Relación ruta-cluster:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Ruta A referencia Cluster X.` → expresa la relación o la condición del ejemplo que se está analizando.
- **Línea 4:** `Ruta B referencia Cluster X.` → expresa la relación o la condición del ejemplo que se está analizando.
- **Línea 5:** `Ruta C referencia Cluster Y.` → expresa la relación o la condición del ejemplo que se está analizando.

La relación se caracteriza por tres propiedades. La primera es que varias rutas pueden apuntar al mismo cluster. La segunda es que la separación permite evolucionar ambos de forma independiente. La tercera es que la relación se expresa con el `ClusterId`. Las tres propiedades permiten que la configuración sea modular.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La referencia ClusterId evita duplicar listas de destinos en cada ruta. Varias reglas pueden compartir un cluster si todas necesitan el mismo servicio y políticas compatibles. Un error en esa referencia debe detectarse al cargar o validar la configuración.

Ejemplo razonado. Materiales y planificaciones comparten el cluster de Inventario. La segunda instancia se añade una vez al cluster final; no se copia su dirección en cada ruta. Las operaciones públicas conservan su URL cuando cambia la topología.

Riesgo, alternativa y límite. Compartir cluster también comparte políticas. Si dos operaciones requieren distintas características de transporte o afinidad, conviene revisar si necesitan conjuntos separados, aun cuando algunas direcciones coincidan. La separación debe responder a un requisito explícito.

Comprobación y aprendizaje. Sigue dos rutas hasta el mismo ClusterId y enumera sus destinos. Después localiza la política de ese cluster. El ejercicio distingue una decisión de publicación de una decisión operativa sin asignar reglas de negocio al gateway.



Interpretación del gráfico: Las dos rutas de la izquierda se relacionan con el mismo cluster de Inventario. Las tarjetas centrales explican la referencia compartida y las de la derecha muestran los destinos disponibles en la evolución del módulo. Añadir la segunda instancia cambia el conjunto de servidores sin cambiar la URL pública. El segundo destino aparece en 8.4; no debe asumirse que está disponible durante las etapas anteriores.

Bloque 5 — Integración de YARP con ASP.NET Core

La integración de YARP con ASP.NET Core se realiza con los métodos `AddReverseProxy` y `MapReverseProxy`. El método `AddReverseProxy` registra los servicios de YARP en el contenedor. El método `MapReverseProxy` añade el middleware de YARP a la aplicación. La configuración se carga desde `appsettings.json` con `LoadFromConfig`. La integración reutiliza el modelo de middleware de ASP.NET Core. La integración permite que YARP se combine con otros middlewares.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

var app = builder.Build();
app.MapReverseProxy();
app.Run();
```

- **Línea 1:** `var builder = WebApplication.CreateBuilder(args);` → crea el constructor.
- **Línea 2:** `builder.Services.AddReverseProxy()` → registra los servicios de YARP.
- **Línea 3:** `.LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));` → carga la configuración.
- **Línea 4:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 5:** `var app = builder.Build();` → construye la aplicación.
- **Línea 6:** `app.MapReverseProxy();` → añade el middleware de YARP.
- **Línea 7:** `app.Run();` → arranca el servidor.

La integración se caracteriza por tres propiedades. La primera es que reutiliza el modelo de middleware de ASP.NET Core. La segunda es que la configuración se carga desde `appsettings.json`. La tercera es que YARP se combina con otros middlewares. Las tres propiedades permiten que el gateway sea coherente con el resto del sistema.

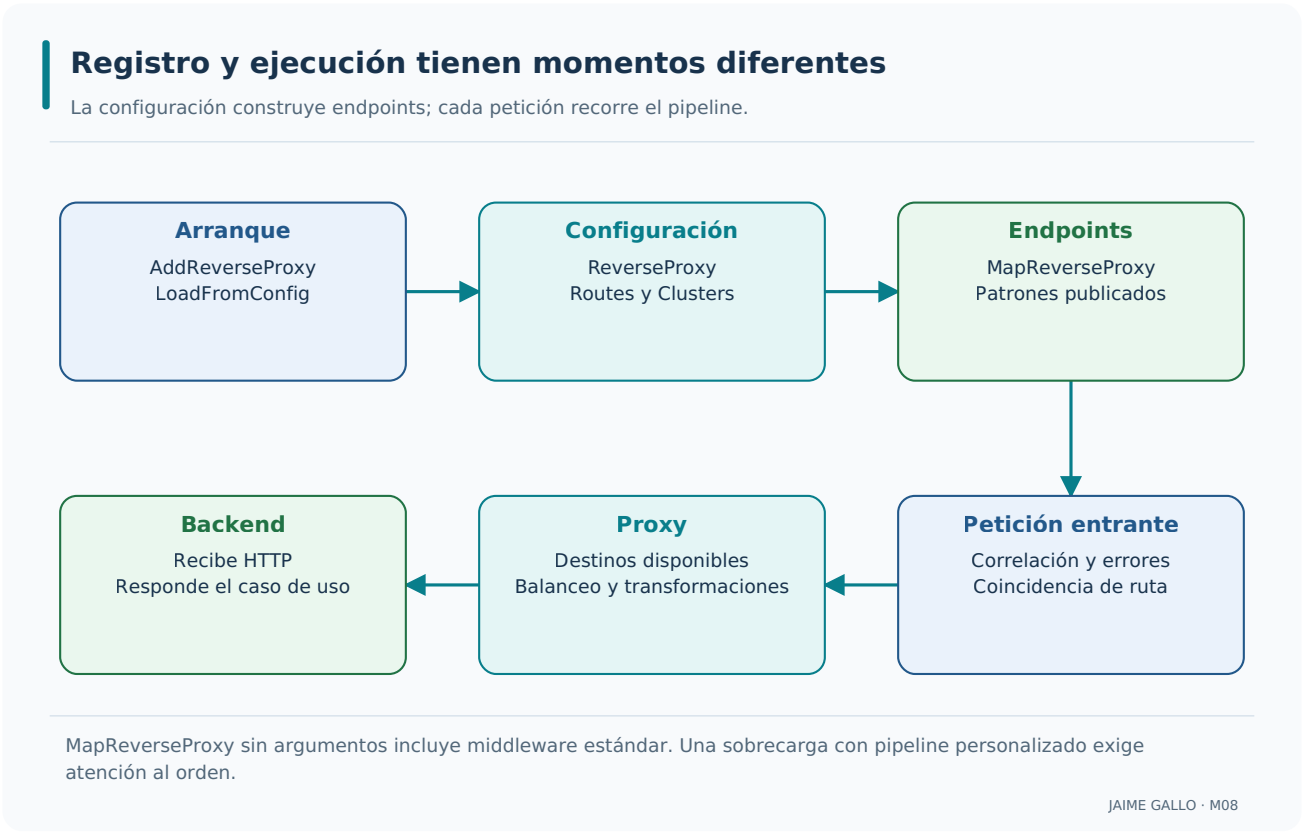
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. AddReverseProxy registra servicios; LoadFromConfig suministra rutas y clusters; MapReverseProxy publica sus endpoints. El orden del middleware exterior determina qué información está disponible antes de reenviar y cómo se observan los resultados.

Ejemplo razonado. La correlación de M07 se establece antes del proxy para que la petición saliente use un valor validado. El resumen HTTP y el tratamiento de errores se integran en el host sin mover dependencias de SQL ni casos de uso al gateway.

Riesgo, alternativa y límite. La sobrecarga sin argumentos de MapReverseProxy incluye el pipeline estándar. Al usar una sobrecarga con configuración de pipeline no debe suponerse que se conservaron todos los componentes. Añadir middleware puede cambiar qué fallos quedan dentro de su alcance.

Comprobación y aprendizaje. Localiza las tres llamadas en Program.cs y distingue qué se ejecuta al arrancar de lo que ocurre por petición. Usa un backend Kestrel controlado para comprobar que el proxy mantiene método, query y cuerpo.



Interpretación del gráfico: La fila superior describe decisiones de arranque y publicación de endpoints; la inferior sigue una petición ya recibida. La correlación precede al envío, el proxy selecciona un destino y el servicio resuelve el caso de uso. Las flechas no indican que AddReverseProxy se ejecute por cada petición. El middleware estándar y las sobrecargas personalizadas requieren una revisión distinta de sus componentes.

Bloque 6 — Estructura de la configuración

La configuración de YARP se organiza en tres secciones principales: `ReverseProxy`, `Routes` y `Clusters`. La sección `ReverseProxy` agrupa las secciones de rutas y clusters. La sección `Routes` declara las rutas. La sección `Clusters` declara los clusters. Cada sección tiene una estructura específica. La estructura se documenta en el archivo `appsettings.json` del proyecto del gateway.

```
{
  "ReverseProxy": {
    "Routes": {
      "ordenes-route": {
        "ClusterId": "ordenes-cluster",
        "Match": {
          "Path": "/api/v1/ordenes/{**catch-all}"
        }
      }
    },
    "Clusters": {
      "ordenes-cluster": {
        "Destinations": {
          "ordenes-primary": {
            "Address": "http://aceria-ordenes:5001/"
          }
        }
      }
    }
  }
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"ReverseProxy": {` → abre la sección de YARP.
- **Línea 3:** `"Routes": {` → abre la sección de rutas.
- **Línea 4:** `"ordenes-route": {` → declara la ruta de órdenes.
- **Línea 5:** `"ClusterId": "ordenes-cluster",` → asocia la ruta al cluster.
- **Línea 6:** `"Match": {` → abre la sección de coincidencia.
- **Línea 7:** `"Path": "/api/v1/ordenes/{**catch-all}"` → patrón de coincidencia.
- **Línea 8:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 9:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 10:** `},` → cierra el objeto abierto en el contexto anterior.
- **Línea 11:** `"Clusters": {` → abre la sección de clusters.
- **Línea 12:** `"ordenes-cluster": {` → declara el cluster de órdenes.
- **Línea 13:** `"Destinations": {` → abre la sección de destinos.
- **Línea 14:** `"ordenes-primary": {` → declara el destino principal.
- **Línea 15:** `"Address": "http://aceria-ordenes:5001/"` → declara una dirección de ejemplo con nombre ASCII; debe resolverse en la red del emisor.
- **Línea 16:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 17:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 18:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 19:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 20:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 21:** `}` → cierra el objeto abierto en el contexto anterior.

La estructura se caracteriza por tres propiedades. La primera es que se organiza en secciones. La segunda es que se documenta. La tercera es que se versiona con el código. Las tres propiedades permiten que la configuración sea coherente y trazable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. ReverseProxy es la sección que se enlaza con IConfiguration. Routes y Clusters son sus colecciones principales; la estructura jerárquica del JSON importa tanto como los nombres. Un fragmento aislado puede ser correcto sin constituir un archivo completo.

Ejemplo razonado. El ejemplo reduce la configuración a una ruta conceptual de órdenes y un destino de su servicio para estudiar la referencia. Las prácticas incluyen la configuración completa de cada etapa, con sus rutas de Producción e Inventario y puertos docentes.

Riesgo, alternativa y límite. La configuración admite actualización cuando la fuente la notifica; eso no vuelve inocuo un cambio de destino. Debe validarse que la revisión cargada publica el mismo contrato y que los nuevos servidores realmente atienden las operaciones esperadas.

Comprobación y aprendizaje. Comprueba que cada ClusterId de Routes existe en Clusters y que cada Address es una URI de backend conocida. Introducir un nombre arbitrario elegido desde una petición abriría una capacidad diferente y peligrosa que el laboratorio no implementa.

Bloque 7 — Transformaciones de YARP

Las transformaciones en YARP permiten modificar la petición antes de enviarla al destino. Las transformaciones de petición se configuran en la ruta y adaptan el mensaje saliente hacia el destino seleccionado. Las transformaciones más usadas son la adición de cabeceras, la eliminación de cabeceras, la modificación de la ruta y la reescritura de la URL. Las transformaciones se configuran en la sección `Transforms` de la ruta. La transformación se expresa con una lista de operaciones.

```
{
  "Transforms": [
    {
      "PathRemovePrefix": "/inventario"
    },
    {
      "RequestHeader": "X-Canal",
      "Set": "Gateway"
    }
  ]
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Transforms": [` → abre la lista de transformaciones.
- **Línea 3:** `{` → abre un objeto en el contexto mostrado.
- **Línea 4:** `"PathRemovePrefix": "/inventario"` → elimina únicamente el prefijo de alias /inventario antes de enviar al backend.
- **Línea 5:** `},` → cierra el objeto abierto en el contexto anterior.
- **Línea 6:** `{` → abre un objeto en el contexto mostrado.
- **Línea 7:** `"RequestHeader": "X-Canal",` → selecciona o establece una cabecera de la petición saliente; el valor mostrado es literal.
- **Línea 8:** `"Set": "Gateway"` → establece el valor literal mostrado; no genera un identificador dinámico.
- **Línea 9:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 10:** `]` → cierra la colección de elementos del contexto mostrado.
- **Línea 11:** `}` → cierra el objeto abierto en el contexto anterior.

Las transformaciones se caracterizan por tres propiedades. La primera es que modifican la petición. La segunda es que se configuran por ruta. La tercera es que se combinan en una lista. Las tres propiedades permiten que el gateway adapte las peticiones al destino.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Las transformaciones de petición se configuran por ruta y modifican el mensaje saliente. La dirección del destino aporta esquema, autoridad y base de ruta; no se usa una transformación genérica para aceptar un host arbitrario del cliente.

Ejemplo razonado. El alias docente elimina /inventario y añade X-Canal con el literal Gateway. La correlación sigue el middleware validado; no se configura como una cadena constante. El recurso interno conserva /api/v1/materiales o /api/v1/planificaciones.

Riesgo, alternativa y límite. PathRemovePrefix debe corresponder a la diferencia real entre contratos. Eliminar /api/v1 de un backend que también espera ese prefijo produciría otra ruta y posiblemente un 404. Una transformación válida sintácticamente puede ser incorrecta funcionalmente.

Comprobación y aprendizaje. Escribe antes y después de una URL concreta y comprueba la petición recibida por el servidor controlado. Compara también la query original y las cabeceras elegidas; cada modificación debe tener un propósito documentado.

Bloque 8 — Configuración por entorno

La configuración de YARP se ajusta por entorno. En desarrollo se usan direcciones locales. En producción se usan direcciones del clúster. La configuración se separa en `appsettings.Development.json` y `appsettings.Production.json`. La configuración por entorno permite que el gateway se adapte a cada contexto. La configuración se documenta en el ADR.

```
Configuración por entorno:

Desarrollo → direcciones locales.
Producción → direcciones del clúster.
```

- **Línea 1:** `Configuración por entorno:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Desarrollo → direcciones locales.` → configuración de desarrollo.
- **Línea 4:** `Producción → direcciones del clúster.` → configuración de producción.

La configuración por entorno se caracteriza por tres propiedades. La primera es que cada entorno tiene su propia configuración. La segunda es que la configuración se separa en archivos. La tercera es que la configuración se versiona. Las tres propiedades permiten que el gateway se adapte a cada contexto.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Los proveedores de configuración permiten variar valores sin recompilar el gateway. Desarrollo y contenedores tienen topologías distintas: localhost significa la máquina del proceso que realiza la llamada, no el equipo del usuario.

Ejemplo razonado. El recorrido local usa 127.0.0.1 y puertos 5105, 5106 y 5107. En una red de contenedores podrían usarse nombres como aceria-inventario; el nombre debe resolverse desde el contenedor del gateway y el puerto debe ser el interno correcto.

Riesgo, alternativa y límite. Una configuración de producción de ejemplo no constituye un despliegue de producción. Certificados, resolución DNS, controles de red, secretos y redundancia requieren decisiones adicionales. No deben incluirse contraseñas dentro del JSON versionado.

Comprobación y aprendizaje. Documenta desde dónde se ejecuta cada proceso y qué dirección usa para hablar con el otro. Una conexión correcta desde el navegador del alumno no prueba resolución desde una red distinta. Comprueba siempre desde el contexto real del emisor.



Interpretación del gráfico: La superficie pública puede conservarse al sustituir direcciones locales por nombres de una red interna. Las tarjetas inferiores añaden las condiciones de una modificación controlada: revisión, comprobación y destinos fijados por el operador. La flecha entre entornos representa una alternativa de despliegue, no un reenvío que pase por ambos. El contexto del proceso emisor determina cómo se resuelve localhost y cada nombre DNS.

Bloque 9 — Buenas prácticas de configuración de YARP

Las buenas prácticas de configuración de YARP son cinco. La primera es usar identificadores descriptivos para rutas y clusters. La segunda es separar la configuración por entorno. La tercera es aplicar transformaciones solo cuando son necesarias. La cuarta es documentar la configuración en el ADR. La quinta es validar la configuración con un test. La aplicación de las cinco prácticas produce una configuración clara y mantenible.

```
Buenas prácticas de configuración de YARP:

1. Identificadores descriptivos.
2. Configuración separada por entorno.
3. Transformaciones solo cuando son necesarias.
4. Documentar en el ADR.
5. Validar con un test.
```

- **Línea 1:** `Buenas prácticas de configuración de YARP:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.

- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `1. Identificadores descriptivos.` → primera práctica.
- **Línea 4:** `2. Configuración separada por entorno.` → segunda práctica.
- **Línea 5:** `3. Transformaciones solo cuando son necesarias.` → tercera práctica.
- **Línea 6:** `4. Documentar en el ADR.` → cuarta práctica.
- **Línea 7:** `5. Validar con un test.` → quinta práctica.

Las buenas prácticas se caracterizan por tres propiedades. La primera es que producen una configuración clara. La segunda es que protegen la coherencia. La tercera es que permiten el análisis automático. Las tres propiedades permiten que la configuración sea un activo del sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Los identificadores descriptivos y un ADR permiten revisar cambios sin deducir intención a partir de direcciones. La comprobación debe abarcar estructura, superficie pública y comportamiento del destino, no limitarse a que el JSON se pueda analizar.

Ejemplo razonado. El catálogo conserva tres clusters conceptuales y comprueba sus indicadores. La configuración ejecutable se ajusta a los dos servicios reales. Esta distinción permite conservar el ejercicio original y, a la vez, aprender qué evidencia falta en una revisión solo estática.

Riesgo, alternativa y límite. Duplicar configuración entre entornos facilita divergencias silenciosas. Mantén claras las opciones que cambian y prueba los caminos importantes después de ajustar direcciones o políticas. No ocultes un fallo cambiando la prueba para que solo inspeccione una cadena.

Comprobación y aprendizaje. Revisa la relación ruta-cluster-destino y después ejecuta una lectura y una escritura publicadas. En una escritura, inspecciona el estado y Location además del cuerpo. Un 201 con enlace inalcanzable incumple una parte del contrato.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica YARP en el módulo 8. El gateway se implementa como un proyecto de ASP.NET Core con YARP. La configuración se realiza en `appsettings.json`. El análisis conserva órdenes, inventario y clientes; las rutas ejecutables publican los recursos existentes de Producción e Inventario. Los clusters apuntan a las direcciones de los servicios. El middleware propaga correlación validada; las transformaciones del alias eliminan su prefijo y añaden un marcador de canal. La validación de la configuración se realiza con un test que verifica la coherencia.

```
Aplicación al proyecto del curso:

Proyecto gateway  → AceriaApp.Gateway.
Configuración     → appsettings.json.
Rutas             → catálogo conceptual; Producción e Inventario ejecutables.
Clusters          → direcciones de los servicios.
Transformaciones  → canal y prefijo de alias; correlación en middleware.
Validación        → test de coherencia.
```

- **Línea 1:** `Aplicación al proyecto del curso:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Proyecto gateway → AceriaApp.Gateway.` → asocia Proyecto gateway con AceriaApp.Gateway, según el alcance conceptual del ejemplo.

- **Línea 4:** `Configuración → appsettings.json.` → asocia Configuración con appsettings.json, según el alcance conceptual del ejemplo.
- **Línea 5:** `Rutas → catálogo conceptual; Producción e Inventario ejecutables.` → asocia Rutas con catálogo conceptual; Producción e Inventario ejecutables, según el alcance conceptual del ejemplo.
- **Línea 6:** `Clusters → direcciones de los servicios.` → asocia Clusters con direcciones de los servicios, según el alcance conceptual del ejemplo.
- **Línea 7:** `Transformaciones → canal y prefijo de alias; correlación en middleware.` → asocia Transformaciones con canal y prefijo de alias; correlación en middleware, según el alcance conceptual del ejemplo.
- **Línea 8:** `Validación → test de coherencia.` → asocia Validación con test de coherencia, según el alcance conceptual del ejemplo.

La aplicación de YARP prepara el proyecto para los puntos 8.3 y 8.4. El punto 8.3 estudiará el enrutamiento de peticiones con detalle. El punto 8.4 estudiará el balanceo de carga básico. El punto 8.2 establece la base que se aplicará en los dos puntos siguientes.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La etapa 8.2 añade el proceso Gateway al acumulado, además de los modelos originales de análisis. El validador conceptual y las pruebas de proxy cumplen propósitos complementarios: intención de configuración y circulación efectiva de HTTP.

Ejemplo razonado. ClusterYarp conserva identificador, destinos y sus dos indicadores. El reto suma tres clusters y tres destinos con configuración por entorno declarada: 3, 3 y 3. Las rutas reales publican los recursos existentes de Producción e Inventario.

Riesgo, alternativa y límite. El servicio Clientes del catálogo no se convierte en una dependencia que haya que arrancar. No debe afirmarse que el gateway docente ofrece autenticación o cuotas porque un modelo las declara. La independencia del proceso también se revisa en sus referencias.

Comprobación y aprendizaje. Comprueba el comportamiento con el backend disponible y detenido. El segundo caso debe comunicar un fallo técnico; nunca debe devolver un objeto de éxito construido por el proxy. Reserva las cifras de tests para los resultados ejecutados sobre la revisión final.

Resumen rápido de la teoría

- YARP es una biblioteca de .NET para construir proxies inversos y gateways.
- Una ruta en YARP define la relación entre una petición entrante y un cluster.
- Un cluster en YARP define el conjunto de destinos.
- La relación entre ruta y cluster es de muchos a uno.
- YARP se integra con ASP.NET Core con `AddReverseProxy` y `MapReverseProxy`.
- La configuración se organiza en las secciones `Routes` y `Clusters`.
- Las transformaciones modifican la petición antes de enviarla al destino.
- La configuración se ajusta por entorno.
- Las buenas prácticas son identificadores descriptivos, configuración por entorno, transformaciones necesarias, documentación en ADR y validación con test.

Punto 8.3 — Enrutamiento de peticiones

Módulo 8: Publicación mediante API Gateway

Objetivos de aprendizaje:

- Comprender el proceso de coincidencia de rutas en YARP.
- Identificar los patrones de ruta y su precedencia.
- Comprender las transformaciones de ruta, cabecera y consulta.
- Comprender el orden de aplicación de las rutas.
- Añadir al proyecto `AceriaApp` las clases de validación del enrutamiento de YARP y pruebas que verifiquen la validación, distinguiéndolas del comportamiento funcional del proxy.

Desarrollo teórico

Bloque 1 — Proceso de coincidencia de rutas

El proceso de coincidencia de rutas determina qué ruta de YARP se aplica a una petición entrante. YARP utiliza endpoint routing para seleccionar una ruta según Order, especificidad y condiciones de coincidencia; no depende de la posición textual en el JSON. La coincidencia se basa en el patrón de la ruta y en condiciones adicionales. Las condiciones incluyen el método HTTP, las cabeceras y los parámetros de consulta. El proceso se ejecuta en cada petición. La evaluación se realiza con un algoritmo de prioridad que ordena las rutas antes de evaluarlas.

```
Proceso de coincidencia:

Petición → condiciones de coincidencia y precedencia → seleccionar endpoint → aplicar ruta.
```

- **Línea 1:** `Proceso de coincidencia:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Petición → condiciones de coincidencia y precedencia → seleccionar endpoint → aplicar ruta.` → asocia Petición con condiciones de coincidencia y precedencia → seleccionar endpoint → aplicar ruta, según el alcance conceptual del ejemplo.

El proceso se caracteriza por tres propiedades. La primera es que las rutas se ordenan por prioridad. La segunda es que se selecciona un endpoint según los criterios aplicables. La tercera es que la evaluación ocurre en cada petición. Las tres propiedades permiten que el enrutamiento sea predecible.

La aplicación del proceso tiene dos consecuencias prácticas. La primera es que el orden de las rutas importa. La segunda es que se aplica el endpoint seleccionado según prioridad y condiciones de coincidencia. La combinación de ambas permite que el enrutamiento sea determinista.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La selección se integra con endpoint routing de ASP.NET Core. No consiste en recorrer propiedades del JSON hasta encontrar la primera coincidencia textual. Se combinan Order, especificidad y condiciones que producen un endpoint válido o un fallo de selección.

Ejemplo razonado. Una consulta de materiales coincide con su superficie publicada y conduce al cluster de Inventario. Una URL fuera de esa superficie debe terminar en el borde sin descubrir un endpoint interno simplemente porque el backend conoce su ruta.

Riesgo, alternativa y límite. Dos reglas igualmente elegibles pueden producir ambigüedad. Cambiar el orden de las propiedades del archivo no es una solución fiable. Debe precisarse el contrato o ajustar Order y condiciones para que la elección sea deliberada.

Comprobación y aprendizaje. Construye una tabla con URL, método y regla esperada. Incluye una petición válida, otra fuera de ruta y una que diferencia condiciones. Confirma en un backend controlado cuál se recibió y conserva la tabla como explicación del ensayo.

Bloque 2 — Patrones de ruta

Un patrón de ruta define la estructura de la URL que la ruta acepta. El patrón se expresa con la sintaxis de rutas de ASP.NET Core. Los segmentos literales coinciden con texto exacto. Los segmentos con parámetros capturan valores. Los segmentos con comodín capturan varios segmentos. El patrón puede incluir restricciones que limitan los valores aceptados. El patrón se declara en la sección `Match.Path` de la ruta.

```
{
  "Match": {
    "Path": "/api/v1/ordenes/{id:guid}",
    "Methods": [ "GET" ]
  }
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Match": {` → abre la sección de coincidencia.
- **Línea 3:** `"Path": "/api/v1/ordenes/{id:guid}",` → patrón de ruta con parámetro y restricción.
- **Línea 4:** `"Methods": [ "GET" ]` → restringe el método HTTP.
- **Línea 5:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 6:** `}` → cierra el objeto abierto en el contexto anterior.

El patrón se caracteriza por tres propiedades. La primera es que usa la sintaxis de ASP.NET Core. La segunda es que puede incluir restricciones. La tercera es que se declara en la configuración. Las tres propiedades permiten que el enrutamiento sea expresivo.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un parámetro captura un segmento; una restricción participa en la coincidencia. El patrón {id:guid} exige una forma compatible con Guid, pero no demuestra que el recurso exista ni que el usuario tenga permiso para leerlo.

Ejemplo razonado. El catálogo original conserva la ruta conceptual de órdenes por id y su catch-all. Las prácticas reales respetan los contratos existentes del acumulado. Los ejemplos de patrones muestran sintaxis y no cambian el tipo de identificador de una API previa.

Riesgo, alternativa y límite. Un catch-all captura varios segmentos y amplía el conjunto de rutas admitidas. El alias de Inventario debe limitarse a materiales y planificaciones; no debe abrir el backend entero con /inventario/{**resto} sin filtro.

Comprobación y aprendizaje. Compara un valor con forma válida, otro que viola una restricción y otro inexistente que sí la cumple. Distingue rechazo de coincidencia y ausencia del recurso: ambos pueden producir 404, pero recorren partes distintas del sistema.

Bloque 3 — Precedencia de rutas

La precedencia de rutas determina qué ruta se evalúa primero cuando varias coinciden. YARP asigna una prioridad a cada ruta basándose en la especificidad del patrón. Las rutas más específicas tienen mayor prioridad. Las rutas más generales tienen menor prioridad. La prioridad se puede ajustar con la propiedad `Order`. Los valores más bajos se evalúan primero. Con el mismo Order, la especificidad contribuye a priorizar rutas concretas; un Order explícito puede cambiar esa elección.

```
Precedencia de rutas:

Ruta específica  → mayor especificidad con el mismo Order.
Ruta general     → menor especificidad con el mismo Order.
```

- **Línea 1:** `Precedencia de rutas:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Ruta específica → mayor especificidad con el mismo Order.` → con el mismo Order, su patrón más específico contribuye a la prioridad; un Order distinto puede cambiar la elección.
- **Línea 4:** `Ruta general → menor especificidad con el mismo Order.` → con el mismo Order, su menor especificidad contribuye a una prioridad inferior; no depende de su posición textual.

La precedencia se caracteriza por tres propiedades. La primera es que se basa en la especificidad. La segunda es que se puede ajustar con `Order`. La tercera es que garantiza el orden de evaluación. Las tres propiedades permiten que el enrutamiento sea predecible.

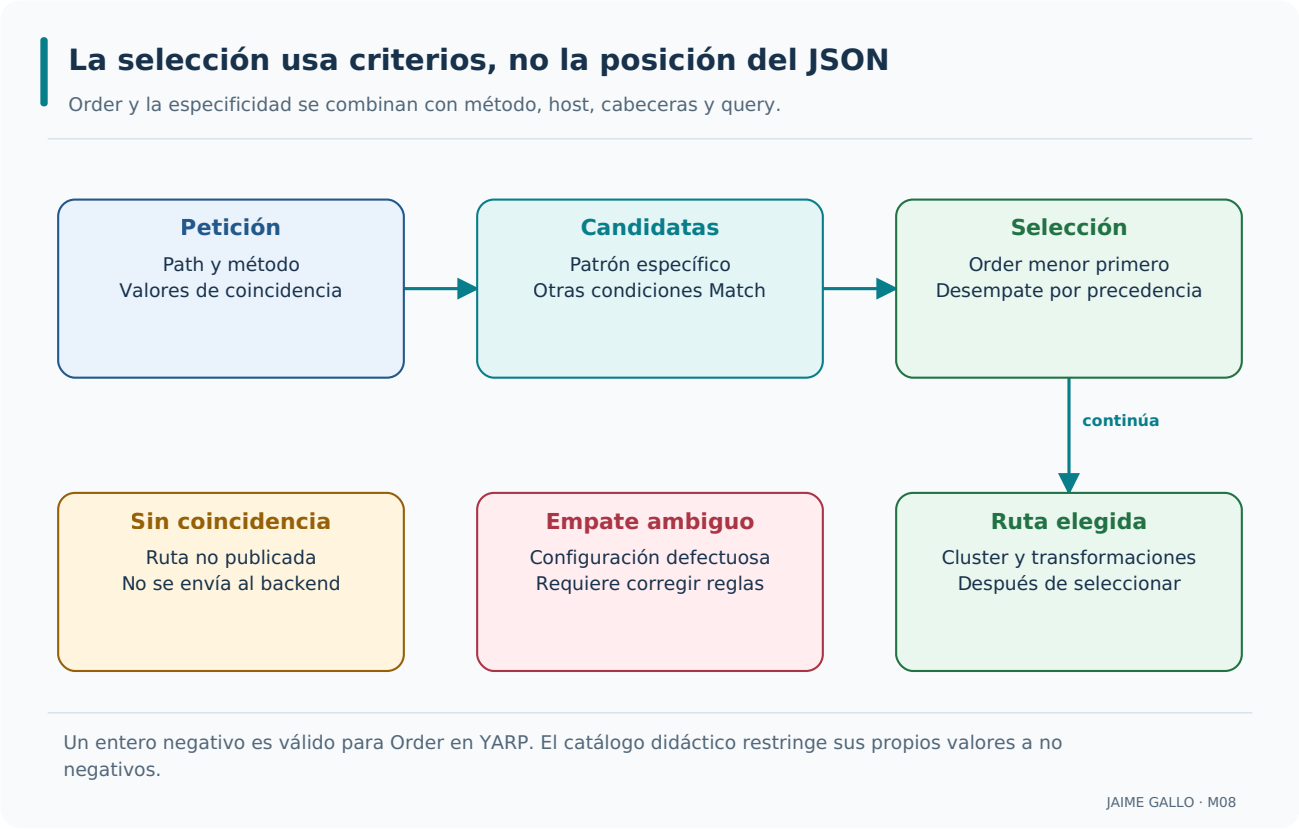
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Order es una prioridad explícita: los valores menores preceden a mayores. Si no se distingue por Order, interviene la especificidad del patrón y otras condiciones. La configuración no debe depender de empates ambiguos.

Ejemplo razonado. El catálogo asigna órdenes 1 a 4 a reglas conceptuales y valida que no sean negativos. Esa limitación es una regla de ese ejercicio; YARP admite valores negativos, por lo que no debe describirse como una restricción del framework.

Riesgo, alternativa y límite. Dar siempre prioridad extrema a una regla general puede eclipsar una específica. Un ejemplo con dos reglas que no se solapan no demuestra precedencia, aunque sus valores estén ordenados. La prueba necesita una petición que pudiera elegir entre ambas.

Comprobación y aprendizaje. Ensaya una coincidencia solapada y registra qué destino recibió la petición. Cambia una prioridad en una copia aislada y explica el efecto. La regla elegida debe deducirse del contrato, no del orden de escritura del JSON.



Interpretación del gráfico: La selección combina criterios antes de aplicar las transformaciones de la ruta elegida. Las dos tarjetas inferiores de la izquierda representan resultados que necesitan tratamiento propio: no coincidencia y ambigüedad. Un valor negativo de Order es válido en YARP, aunque el catálogo didáctico lo rechace por una política local. La posición textual del JSON no es el mecanismo de selección y no resuelve un empate.

Bloque 4 — Transformaciones de ruta

Las transformaciones de ruta modifican la ruta de la petición antes de enviarla al destino. La transformación `PathRemovePrefix` elimina un prefijo de la ruta. La transformación `PathSet` establece una ruta fija. La transformación `PathPattern` aplica un patrón a la ruta. Las transformaciones de ruta se aplican después de la coincidencia. Las transformaciones permiten que la ruta externa y la ruta interna sean distintas.

```
{
  "Transforms": [
    { "PathRemovePrefix": "/inventario" }
  ]
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Transforms": [` → abre la lista de transformaciones.
- **Línea 3:** `{ "PathRemovePrefix": "/inventario" }` → elimina únicamente el prefijo de alias / inventario antes de enviar al backend.
- **Línea 4:** `]` → cierra la colección de elementos del contexto mostrado.
- **Línea 5:** `}` → cierra el objeto abierto en el contexto anterior.

Las transformaciones de ruta se caracterizan por tres propiedades. La primera es que modifican la ruta de la petición. La segunda es que se aplican después de la coincidencia. La tercera es que se combinan en una lista. Las tres propiedades permiten que el enrutamiento sea flexible.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La coincidencia usa la URL de entrada. La transformación se aplica después para construir la petición saliente. Esta separación permite un alias público sin obligar al backend a conocer ese alias.

Ejemplo razonado. El ejemplo /inventario/api/v1/materiales elimina solo /inventario y llega como /api/v1/materiales. PathSet reemplazaría la ruta; PathPattern la construiría con valores capturados. Son operaciones distintas y no deben usarse de forma intercambiable.

Riesgo, alternativa y límite. Una transformación de petición no arregla por sí sola Location de una respuesta. Si una creación devuelve un enlace interno, el borde debe asegurar que el cliente reciba una URL pública válida sin alterar la identidad del recurso creado.

Comprobación y aprendizaje. Anota URL entrante, prefijo eliminado y ruta resultante. Comprueba el valor recibido por el servidor y sigue cualquier Location devuelto al crear. Así se comprueba el contrato en ambas direcciones.

Bloque 5 — Transformaciones de cabecera

Las transformaciones de cabecera modifican las cabeceras de la petición. La transformación `RequestHeader` modifica una cabecera de la petición. La transformación `ResponseHeader` modifica una cabecera de la respuesta. Set reemplaza y Append añade; la eliminación usa claves específicas como RequestHeaderRemove o ResponseHeaderRemove. La transformación `Set` reemplaza el valor de la cabecera. La transformación `Append` añade un valor. RequestHeaderRemove elimina una cabecera de petición y ResponseHeaderRemove una de respuesta. Las transformaciones de cabecera permiten añadir información de contexto a la petición.

```
{
  "Transforms": [
    { "RequestHeader": "X-Canal", "Set": "Gateway" },
    { "RequestHeader": "X-Forwarded-Host", "Set": "aceria.local" }
  ]
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Transforms": [` → abre la lista de transformaciones.
- **Línea 3:** `{ "RequestHeader": "X-Canal", "Set": "Gateway" },` → selecciona o establece una cabecera de la petición saliente; el valor mostrado es literal.
- **Línea 4:** `{ "RequestHeader": "X-Forwarded-Host", "Set": "aceria.local" }` → selecciona o establece una cabecera de la petición saliente; el valor mostrado es literal.
- **Línea 5:** `]` → cierra la colección de elementos del contexto mostrado.
- **Línea 6:** `}` → cierra el objeto abierto en el contexto anterior.

Las transformaciones de cabecera se caracterizan por tres propiedades. La primera es que modifican las cabeceras. La segunda es que se aplican por ruta. La tercera es que se combinan en una lista. Las tres propiedades permiten que el enrutamiento añada contexto.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Set reemplaza un valor y Append agrega otro. Para quitar una cabecera de petición existe RequestHeaderRemove; no se configura una operación Remove genérica dentro de RequestHeader. La respuesta tiene sus transformaciones específicas.

Ejemplo razonado. X-Canal=Gateway es deliberadamente constante y permite reconocer el alias en una prueba. X-Correlation-ID debe ser un valor efectivo por petición: se valida o genera antes del envío y se conserva entre procesos.

Riesgo, alternativa y límite. Un cliente puede enviar su propia X-Canal. Reemplazarla evita confundirla con la marca elegida para el ejemplo, pero no convierte esa marca en credencial. Tampoco deben copiarse ciegamente Host o cabeceras de reenvío recibidas de un origen no confiable.

Comprobación y aprendizaje. Envía una X-Canal distinta y comprueba el valor que recibe el backend. Haz un ensayo separado para correlación y revisa que la sustitución de un valor inválido se haga una sola vez en el borde.

Bloque 6 — Transformaciones de consulta

Las transformaciones de consulta modifican los parámetros de consulta de la petición. La transformación QueryRouteParameter toma un valor capturado de la ruta y lo establece o añade como parámetro de consulta. QueryValueParameter establece o añade un valor literal. Copiar una cabecera a la consulta exige una transformación personalizada en código. Las transformaciones de consulta permiten que el destino reciba información que estaba en la ruta o en la cabecera. Su posición relativa depende de la lista Transforms de cada ruta.

```
{
  "Transforms": [
    { "QueryRouteParameter": "id", "Set": "id" },
    { "QueryValueParameter": "canal", "Set": "gateway" }
  ]
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Transforms": [` → abre la lista de transformaciones.
- **Línea 3:** `{ "QueryRouteParameter": "id", "Set": "id" },` → obtiene el valor capturado id y lo establece en la consulta saliente.
- **Línea 4:** `{ "QueryValueParameter": "canal", "Set": "gateway" }` → establece el marcador literal canal=gateway en la consulta saliente.
- **Línea 5:** `]` → cierra la colección de elementos del contexto mostrado.
- **Línea 6:** `}` → cierra el objeto abierto en el contexto anterior.

Las transformaciones de consulta se caracterizan por tres propiedades. La primera es que añaden parámetros de consulta. La segunda es que usan valores literales o de ruta; una cabecera requiere código personalizado. La tercera es que se combinan en una lista. Las tres propiedades permiten que el enrutamiento enriquezca la petición.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. QueryValueParameter establece o añade un valor literal; QueryRouteParameter obtiene el valor de un parámetro capturado por la ruta. Copiar una cabecera a query requiere una transformación de código y una revisión de qué datos pueden exponerse.

Ejemplo razonado. El ejemplo usa QueryRouteParameter para id y QueryValueParameter para canal=gateway. El alias funcional utiliza el segundo caso y conserva los parámetros originales. No depende de una clave inexistente llamada QueryParameterFromHeader.

Riesgo, alternativa y límite. Mover datos de una cabecera a query cambia su visibilidad: las URL pueden aparecer en históricos o registros de intermediarios. La transformación no debe emplearse para desplazar secretos o tokens. Un marcador de canal no es información de identidad.

Comprobación y aprendizaje. Prueba una query original como pagina=1 y verifica su conservación junto con canal=gateway. Para un valor capturado, comprueba la codificación de caracteres y el nombre exacto de la clave, sin construir URL mediante concatenación insegura.

Bloque 7 — Orden de aplicación de las transformaciones

El orden de aplicación de las transformaciones determina el resultado final. Las transformaciones se aplican en el orden en que aparecen en la lista. No se impone una fase general de ruta anterior a todas las cabeceras. Tampoco se reordenan todas las cabeceras antes de la consulta: se respeta la lista configurada. El orden determina qué valores están disponibles en cada transformación. El orden se documenta en la configuración. El orden se valida con un test.

```
Orden de aplicación:

1. Leer la lista Transforms de la ruta.
2. Aplicar sus operaciones en el orden declarado.
3. Comprobar el mensaje resultante; no hay reordenación por tipo.
```

- **Línea 1:** `Orden de aplicación:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `1. Leer la lista Transforms de la ruta.` → identifica una decisión de la secuencia o de la lista de comprobación mostrada.
- **Línea 4:** `2. Aplicar sus operaciones en el orden declarado.` → identifica una decisión de la secuencia o de la lista de comprobación mostrada.
- **Línea 5:** `3. Comprobar el mensaje resultante; no hay reordenación por tipo.` → identifica una decisión de la secuencia o de la lista de comprobación mostrada.

El orden se caracteriza por tres propiedades. La primera es que las transformaciones se aplican en orden. La segunda es que el orden determina el resultado. La tercera es que el orden se documenta. Las tres propiedades permiten que el enrutamiento sea predecible.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La lista Transforms expresa la secuencia. No existe un reordenamiento universal que ejecute primero todos los cambios de ruta, luego cabeceras y después query. Las dependencias entre operaciones deben quedar claras en esa lista.

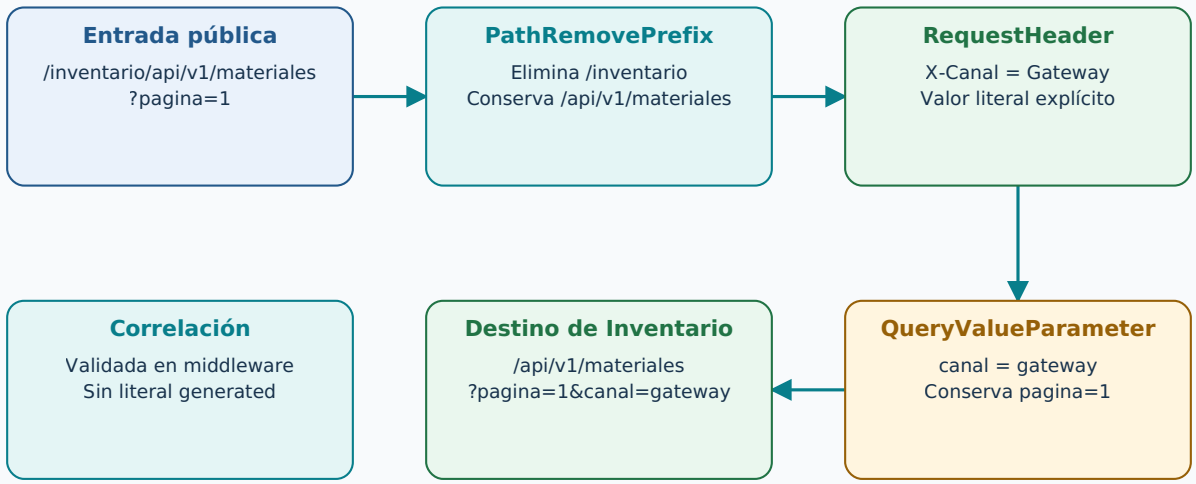
Ejemplo razonado. En el alias se elimina el prefijo, se establece X-Canal y se añade canal. Ese orden coincide con el ejemplo elegido; no constituye una regla general impuesta por YARP. Dos cambios sobre el mismo campo pueden dar resultados distintos al invertirse.

Riesgo, alternativa y límite. Una transformación de respuesta pertenece a otra fase y no debe mezclarse conceptualmente con la preparación de la petición. Además, editar la ruta no vuelve a ejecutar la selección del endpoint original como si hubiera llegado una petición nueva.

Comprobación y aprendizaje. Compara la lista con la petición observada por Kestrel. Para estudiar dependencia, usa una copia controlada con dos operaciones sobre la misma ruta. Documenta la secuencia exacta que produce el resultado esperado.

Un alias conserva el recurso y adapta el camino

Ejemplo docente: /inventario/api/v1/materiales?pagina=1.



Las transformaciones se aplican según la lista configurada. Una cabecera fija de canal no identifica ni autentica al cliente.

JAIME GALLO · M08

Interpretación del gráfico: El recorrido parte de una URL que ya coincide con una ruta publicada. PathRemovePrefix quita únicamente /inventario; la query conserva pagina=1 y añade el marcador de canal. La correlación se representa aparte porque se valida en middleware y no se convierte en un literal de configuración. Las flechas reproducen el orden de este ejemplo, sin imponer una secuencia universal por tipo de transformación.

Bloque 8 — Enrutamiento basado en cabeceras

El enrutamiento basado en cabeceras permite que la ruta se seleccione según el valor de una cabecera. La condición puede estudiar cabeceras como Accept o una personalizada; Hosts ofrece su propio criterio para el host. Coincidir con Authorization no autentica al cliente. La ruta incluye una condición adicional en la sección `Match`. La condición se expresa con `Headers`. La coincidencia se realiza solo si la cabecera tiene el valor esperado. El enrutamiento por cabecera permite variantes por cliente sin duplicar rutas.

```
{
  "Match": {
    "Path": "/api/v1/ordenes/{**catch-all}",
    "Headers": [
      { "Name": "X-Client", "Values": [ "mobile" ] }
    ]
  }
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Match": {` → abre la sección de coincidencia.
- **Línea 3:** `"Path": "/api/v1/ordenes/{**catch-all}",` → patrón de ruta.

- **Línea 4:** `"Headers": [` → abre la lista de condiciones de cabecera.
- **Línea 5:** `{ "Name": "X-Client", "Values": [ "mobile" ] }` → condición de cabecera.
- **Línea 6:** `]` → cierra la colección de elementos del contexto mostrado.
- **Línea 7:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 8:** `}` → cierra el objeto abierto en el contexto anterior.

El enrutamiento por cabecera se caracteriza por tres propiedades. La primera es que la ruta se selecciona según la cabecera. La segunda es que la condición se declara en `Match`. La tercera es que permite variantes por cliente. Las tres propiedades permiten que el enrutamiento sea flexible.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Headers permite condiciones adicionales de coincidencia. Hosts tiene su propio criterio de configuración; una condición sobre Authorization no verifica la firma ni los permisos de un token. Enrutamiento y autenticación responden a cuestiones diferentes.

Ejemplo razonado. El ejemplo X-Client=mobile muestra selección por un valor de cabecera. Es un caso conceptual para comprender Match.Headers; no se introduce una aplicación móvil o una identidad nueva en el proyecto acumulativo.

Riesgo, alternativa y límite. Un valor controlado por el cliente puede cambiar qué regla coincide. Si se usa para aislar versiones o inquilinos, hacen falta políticas que comprueben que el cliente está autorizado a esa selección. La simple coincidencia no ofrece esa garantía.

Comprobación y aprendizaje. Envía el valor esperado, un valor distinto y ninguna cabecera. Compara las reglas elegibles en los tres casos. No uses un resultado de coincidencia como evidencia de que se haya autenticado a la persona.

Bloque 9 — Buenas prácticas de enrutamiento

Las buenas prácticas de enrutamiento son cinco. La primera es aprovechar la especificidad cuando las rutas tienen el mismo Order. La segunda es usar restricciones en los parámetros. La tercera es aplicar transformaciones solo cuando son necesarias. La cuarta es documentar el orden de las rutas. La quinta es validar el enrutamiento con un test. La aplicación de las cinco prácticas produce un enrutamiento claro y predecible.

```
Buenas prácticas de enrutamiento:

1. Con el mismo Order, aprovechar la especificidad de las rutas.
2. Restricciones en los parámetros.
3. Transformaciones solo cuando son necesarias.
4. Documentar el orden de las rutas.
5. Validar con un test.
```

- **Línea 1:** `Buenas prácticas de enrutamiento:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `1. Con el mismo Order, aprovechar la especificidad de las rutas.` → usa la especificidad como criterio de precedencia sin ignorar una prioridad Order explícita.
- **Línea 4:** `2. Restricciones en los parámetros.` → segunda práctica.
- **Línea 5:** `3. Transformaciones solo cuando son necesarias.` → tercera práctica.
- **Línea 6:** `4. Documentar el orden de las rutas.` → cuarta práctica.

- **Línea 7:** `5. Validar con un test.` → quinta práctica.

Las buenas prácticas se caracterizan por tres propiedades. La primera es que producen un enrutamiento claro. La segunda es que protegen la coherencia. La tercera es que permiten el análisis automático. Las tres propiedades permiten que el enrutamiento sea un activo del sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Las buenas prácticas se traducen en casos de prueba concretos. Especificidad, restricciones y prioridades deben evitar ambigüedades; las transformaciones deben conservar la semántica del recurso y justificar cada cambio.

Ejemplo razonado. Las pruebas del catálogo verifican cuatro reglas ordenadas, con ruta y cabecera declaradas en todas y consulta en dos. Las pruebas funcionales verifican que la URL pública, método y mensaje saliente producen el resultado esperado en un servidor real.

Riesgo, alternativa y límite. Tener siempre un flag de transformación true no es un requisito de YARP. Una ruta puede reenviarse sin modificar nada. Mantener esa exigencia en el validador original sirve al ejercicio, pero no debe obligar a añadir modificaciones inútiles al proxy.

Comprobación y aprendizaje. Incluye éxito, método no admitido, prefijo desconocido y error del backend en la revisión. Inspecciona Location en creaciones y verifica que un fallo no se convierte en éxito. Esos casos prueban decisiones diferentes del contrato.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica el enrutamiento en el módulo 8. Las rutas se seleccionan combinando Order, especificidad y condiciones; con el mismo Order, la especificidad contribuye a la prioridad. Las transformaciones de ruta del alias eliminan /inventario y conservan el prefijo /api/v1 esperado por el backend. Las transformaciones de cabecera añaden X-Canal=Gateway; el middleware conserva la correlación validada. Las transformaciones de consulta del laboratorio añaden canal=gateway como valor literal. La validación del enrutamiento se realiza con un test que verifica la coherencia.

```
Aplicación al proyecto del curso:

Orden de rutas       → Order y especificidad; evitar ambigüedad.
Transformaciones ruta → eliminar prefijo.
Transformaciones cabecera → X-Canal=Gateway; correlación mediante middleware.
Transformaciones consulta → canal=gateway como valor literal.
Validación           → test de coherencia.
```

- **Línea 1:** `Aplicación al proyecto del curso:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Orden de rutas → Order y especificidad; evitar ambigüedad.` → combina Order y especificidad, con condiciones suficientes para evitar una selección ambigua.
- **Línea 4:** `Transformaciones ruta → eliminar prefijo.` → asocia Transformaciones ruta con eliminar prefijo, según el alcance conceptual del ejemplo.
- **Línea 5:** `Transformaciones cabecera → X-Canal=Gateway; correlación mediante middleware.` → asocia Transformaciones cabecera con X-Canal=Gateway; correlación mediante middleware, según el alcance conceptual del ejemplo.
- **Línea 6:** `Transformaciones consulta → canal=gateway como valor literal.` → asocia Transformaciones consulta con canal=gateway como valor literal, según el alcance conceptual del ejemplo.

- **Línea 7:** `Validación → test de coherencia.` → asocia Validación con test de coherencia, según el alcance conceptual del ejemplo.

La aplicación del enrutamiento prepara el proyecto para el punto 8.4. El punto 8.4 estudiará el balanceo de carga básico. El punto 8.3 establece la base que se aplicará en el punto 8.4.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La etapa 8.3 conserva ReglaEnrutamiento y sus validaciones y añade comportamiento real del alias. El objetivo es explicar cómo una regla declarada termina en una petición HTTP verificable, sin mezclar conceptos de catálogo y endpoint.

Ejemplo razonado. El reto cuenta cuatro reglas, cuatro con transformación de ruta, cuatro con cabecera y dos con consulta. El resultado 4,4,4,2 describe los indicadores del catálogo y no significa que se hayan enviado cuatro peticiones de red.

Riesgo, alternativa y límite. El prefijo eliminado en la integración es /inventario, no /api/v1. Se añade un literal de canal y se propaga correlación validada. Los endpoints internos continúan fuera de la superficie pública aun cuando compartan backend.

Comprobación y aprendizaje. Ejecuta el alias con un recurso publicado y comprueba que llega al mismo recurso interno. Después intenta una ruta interna no publicada. La primera prueba acredita adaptación; la segunda comprueba que el alias no se volvió un acceso irrestricto.



Interpretación del gráfico: Las tres columnas presentan aspectos paralelos: resultado funcional, error de contrato y fallo de transporte. Cada uno tiene comprobaciones distintas, por lo que un único test de estado 200 resulta insuficiente. Seguir Location y comprobar la correlación completan el análisis de una respuesta válida. Delimitar métodos y superficie protege el contrato de publicación; no equivale a instalar un sistema de autorización.

Resumen rápido de la teoría

- El proceso de coincidencia de rutas determina qué ruta se aplica a una petición.

- Los patrones de ruta definen la estructura de la URL que la ruta acepta.
- La precedencia de rutas determina el orden de evaluación.
- Las transformaciones de ruta modifican la ruta de la petición.
- Las transformaciones de cabecera modifican las cabeceras.
- Las transformaciones de consulta modifican los parámetros de consulta.
- El orden de aplicación de las transformaciones determina el resultado final.
- El enrutamiento basado en cabeceras permite variantes por cliente.
- Las buenas prácticas son prioridades explícitas cuando se necesitan, especificidad sin ambigüedad, restricciones en parámetros, transformaciones necesarias, documentación del orden y validación con test.

Punto 8.4 — Balanceo de carga básico

Módulo 8: Publicación mediante API Gateway

Objetivos de aprendizaje:

- Comprender qué es el balanceo de carga y su papel en una arquitectura de microservicios.
- Identificar los algoritmos de balanceo soportados por YARP.
- Comprender la configuración de destinos múltiples y pesos.
- Comprender la integración del balanceo con los health checks.
- Añadir al proyecto `AceriaApp` las clases de validación de balanceo de carga y pruebas que verifiquen la validación, distinguiéndolas del comportamiento funcional del proxy. Cerrar el módulo 8 y el curso con el resumen final del estado del proyecto.

Desarrollo teórico

Bloque 1 — Definición de balanceo de carga

El balanceo de carga es la distribución del tráfico entre varias instancias de un servicio. Su propósito es repartir la carga para evitar la saturación de una única instancia. El balanceo se realiza en el gateway o en un componente dedicado. El balanceo permite que el sistema escale horizontalmente. La distribución se realiza según un algoritmo que decide qué instancia recibe cada petición. El balanceo es la base de la escalabilidad de los sistemas distribuidos.

```
Balanceo de carga:

Petición → Balanceador → Instancia 1.
                       → Instancia 2.
                       → Instancia 3.
```

- **Línea 1:** `Balanceo de carga:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Petición → Balanceador → Instancia 1.` → asocia Petición con Balanceador → Instancia 1, según el alcance conceptual del ejemplo.
- **Línea 4:** `→ Instancia 2.` → segunda instancia.
- **Línea 5:** `→ Instancia 3.` → tercera instancia.

El balanceo de carga se caracteriza por tres propiedades. La primera es que distribuye el tráfico entre varias instancias. La segunda es que permite el escalado horizontal. La tercera es que se basa en un algoritmo de distribución. Las tres propiedades permiten que el sistema sea escalable y tolerante a fallos.

La aplicación del balanceo tiene dos consecuencias prácticas. La primera es que la carga se reparte entre las instancias. La segunda es que, tras detectar el fallo de una instancia, las nuevas solicitudes pueden continuar por otras si conservan capacidad y dependencias disponibles. La combinación de ambas permite que el sistema sea escalable y resiliente.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Balancear significa elegir un destino para cada solicitud entre candidatos elegibles. No implica replicar peticiones ni crear réplicas. La disponibilidad del conjunto y la compatibilidad funcional de sus miembros preceden a cualquier algoritmo.

Ejemplo razonado. La etapa final mantiene un gateway en 5100 y dos procesos Inventario en 5106 y 5107. Las lecturas conservan el contrato; una identidad técnica de instancia permite observar el reparto sin alterar las reglas del dominio.

Riesgo, alternativa y límite. La caída de una dependencia compartida puede inutilizar ambas instancias. Del mismo modo, conservar sesión exclusivamente en memoria de una instancia puede hacer que una solicitud posterior falle al ir a otra. El balanceador no sincroniza ese estado.

Comprobación y aprendizaje. Envía varias consultas y observa las identidades que contestan. Después compara sus cuerpos funcionales y la persistencia tras reiniciar un proceso. El reparto debe conservar significado y datos, además de alternar emisores.

Bloque 2 — Algoritmos de balanceo

Existen varios algoritmos de balanceo de carga. El algoritmo round-robin distribuye las peticiones de forma cíclica entre las instancias. El algoritmo random elige una instancia de forma aleatoria. El algoritmo least-requests elige la instancia con menos peticiones activas. El algoritmo power-of-two-choices elige la mejor entre dos instancias aleatorias. El algoritmo de peso distribuye según el peso asignado a cada instancia. Cada algoritmo tiene ventajas e inconvenientes según el contexto.

```
Algoritmos de balanceo:

RoundRobin          → distribución cíclica.
Random              → distribución aleatoria.
LeastRequests       → instancia con menos peticiones.
PowerOfTwoChoices   → mejor entre dos aleatorias.
Peso                → distribución según peso.
```

- **Línea 1:** `Algoritmos de balanceo:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `RoundRobin → distribución cíclica.` → asocia RoundRobin con distribución cíclica, según el alcance conceptual del ejemplo.
- **Línea 4:** `Random → distribución aleatoria.` → asocia Random con distribución aleatoria, según el alcance conceptual del ejemplo.
- **Línea 5:** `LeastRequests → instancia con menos peticiones.` → asocia LeastRequests con instancia con menos peticiones, según el alcance conceptual del ejemplo.
- **Línea 6:** `PowerOfTwoChoices → mejor entre dos aleatorias.` → asocia PowerOfTwoChoices con mejor entre dos aleatorias, según el alcance conceptual del ejemplo.

- **Línea 7:** `Peso → distribución según peso.` → asocia Peso con distribución según peso, según el alcance conceptual del ejemplo.

Los algoritmos se caracterizan por tres propiedades. La primera es que cada algoritmo tiene un propósito específico. La segunda es que la elección depende del contexto. La tercera es que el algoritmo se configura por cluster. Las tres propiedades permiten que el balanceo se adapte a cada situación.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. RoundRobin recorre candidatos; Random selecciona al azar; LeastRequests compara solicitudes activas; PowerOfTwoChoices limita la comparación a dos candidatos aleatorios. Ninguna de estas políticas mide automáticamente la capacidad real de CPU.

Ejemplo razonado. AceriaApp usa RoundRobin explícito para que el reparto resulte observable. El catálogo original también muestra PowerOfTwoChoices y LeastRequests para comparar decisiones. El balanceo ponderado se conserva como concepto y exige una implementación adicional en YARP.

Riesgo, alternativa y límite. Un resultado de una muestra pequeña no demuestra equilibrio estadístico de una política aleatoria. Tampoco una alternancia de asignación garantiza orden de finalización: dos backend pueden responder a velocidades diferentes y los clientes pueden concurrir.

Comprobación y aprendizaje. Identifica la política configurada y el conjunto disponible antes de interpretar la secuencia. Para el ensayo docente usa solicitudes controladas y registra las identidades; evita deducir un porcentaje de capacidad a partir de dos respuestas.

Bloque 3 — Configuración de destinos múltiples

La configuración de destinos múltiples se realiza en la sección `Destinations` del cluster. Cada destino tiene un identificador único y una dirección. La lista de destinos puede contener una o varias entradas. El balanceo distribuye las peticiones entre los destinos según el algoritmo configurado. Una política personalizada puede interpretar pesos en metadatos; las políticas incorporadas no lo hacen por declarar una propiedad adicional. La configuración se documenta en el archivo `appsettings.json`.

```
{
  "Clusters": {
    "ordenes-cluster": {
      "Destinations": {
        "ordenes-1": { "Address": "http://aceria-ordenes-1:5001/" },
        "ordenes-2": { "Address": "http://aceria-ordenes-2:5001/" },
        "ordenes-3": { "Address": "http://aceria-ordenes-3:5001/" }
      }
    }
  }
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Clusters": {` → abre la sección de clusters.
- **Línea 3:** `"ordenes-cluster": {` → declara el cluster de órdenes.
- **Línea 4:** `"Destinations": {` → abre la sección de destinos.
- **Línea 5:** `"ordenes-1": { "Address": "http://aceria-ordenes-1:5001/" },` → declara una dirección de ejemplo con nombre ASCII; debe resolverse en la red del emisor.
- **Línea 6:** `"ordenes-2": { "Address": "http://aceria-ordenes-2:5001/" },` → declara una dirección de ejemplo con nombre ASCII; debe resolverse en la red del emisor.
- **Línea 7:** `"ordenes-3": { "Address": "http://aceria-ordenes-3:5001/" }` → declara una dirección de ejemplo con nombre ASCII; debe resolverse en la red del emisor.

- **Línea 8:** } → cierra el objeto abierto en el contexto anterior.
- **Línea 9:** } → cierra el objeto abierto en el contexto anterior.
- **Línea 10:** } → cierra el objeto abierto en el contexto anterior.
- **Línea 11:** } → cierra el objeto abierto en el contexto anterior.

La configuración de destinos múltiples se caracteriza por tres propiedades. La primera es que cada destino tiene un identificador único. La segunda es que la dirección puede variar por instancia. La tercera es que los destinos se configuran por cluster. Las tres propiedades permiten que el balanceo sea configurable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Destinations declara identificadores y direcciones. Tener dos entradas solo crea dos candidatos de configuración; deben existir procesos escuchando y compartir un contrato compatible. La política de salud puede reducir el conjunto efectivo.

Ejemplo razonado. Los destinos reales de Inventario son http://127.0.0.1:5106/ y http://127.0.0.1:5107/. Se conserva Producción en su cluster. No se añade una réplica de Clientes para satisfacer una cifra del catálogo conceptual.

Riesgo, alternativa y límite. Direcciones duplicadas con nombres distintos no producen redundancia real. Dos procesos en la misma máquina tampoco protegen frente a caída del equipo. El laboratorio estudia la pérdida de proceso, y sus conclusiones no deben extenderse a todos los fallos posibles.

Comprobación y aprendizaje. Arranca ambas instancias, comprueba readiness de cada una y consulta la identidad a través del gateway. Luego detén una y verifica la transición tras la detección. Una lista con dos URL no basta para acreditar esta secuencia.



Interpretación del gráfico: El cluster decide una asignación por solicitud y las sucesivas asignaciones recorren los destinos elegibles. Las flechas que salen del cluster representan destinos alternativos; no son un envío simultáneo ni una llamada entre réplicas. Ambas dependen del estado persistente del servicio. La identidad técnica observada permite demostrar reparto sin convertir el número de instancia en una regla de negocio ni en una garantía de alta disponibilidad global.

Bloque 4 — Pesos de destino

Los pesos de destino permiten distribuir la carga de forma ponderada. En una política ponderada adecuada, un peso doble expresa una proporción objetivo 2:1 sobre suficientes peticiones y destinos disponibles. La ponderación se usa cuando las instancias tienen capacidades distintas. YARP 2.2.0 no incluye una propiedad Weight en DestinationConfig; la ponderación necesita una implementación de ILoadBalancingPolicy que interprete metadatos. La suma de los pesos determina la proporción de cada instancia. La ponderación se documenta en la configuración.

```
{
  "Destinations": {
    "ordenes-1": { "Address": "http://aceria-ordenes-1:5001/", "Metadata": { "peso-conceptual":
"2" } },
    "ordenes-2": { "Address": "http://aceria-ordenes-2:5001/", "Metadata": { "peso-conceptual":
"1" } }
  }
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Destinations": {` → abre la sección de destinos.
- **Línea 3:** `"ordenes-1": { "Address": "http://aceria-ordenes-1:5001/", "Metadata": { "peso-conceptual": "2" } },` → declara metadatos conceptuales; una política personalizada tendría que interpretarlos.

- **Línea 4:** `"ordenes-2": { "Address": "http://aceria-ordenes-2:5001/", "Metadata": { "peso-conceptual": "1" } }` → declara metadatos conceptuales; una política personalizada tendría que interpretarlos.
- **Línea 5:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 6:** `}` → cierra el objeto abierto en el contexto anterior.

Los pesos se caracterizan por tres propiedades. La primera es que permiten distribuir de forma ponderada. La segunda es que se usan con instancias de distinta capacidad. La tercera es que se configuran por destino. Las tres propiedades permiten que el balanceo se adapte a la capacidad.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Un peso expresa una proporción deseada, por ejemplo 2:1 bajo condiciones estables. Su cumplimiento depende del algoritmo, del conjunto sano y del número de solicitudes. No es una promesa de latencia ni de uso de CPU.

Ejemplo razonado. El catálogo conserva TienePesos para estudiar esa intención. El JSON del ejemplo emplea Metadata con una clave de peso conceptual; una política personalizada tendría que leerla y aplicarla. Las políticas incorporadas no lo hacen por añadir esa clave.

Riesgo, alternativa y límite. DestinationConfig no define Weight. Publicar esa propiedad como nativa enseñaría una capacidad inexistente. El laboratorio funcional usa RoundRobin sin ponderación; desarrollar una política ponderada queda como ampliación arquitectónica separada.

Comprobación y aprendizaje. Distingue tres niveles: proporción explicada, metadato presente y algoritmo implementado. Solo el tercero puede acreditar reparto ponderado. Si se implementara, harían falta pruebas con fallos de destinos y muestras suficientes, además de la prueba de configuración.

Bloque 5 — Algoritmo de balanceo en YARP

YARP soporta varios algoritmos de balanceo con la propiedad `LoadBalancingPolicy`. La política se configura en el cluster. Los valores incorporados incluyen RoundRobin, Random, LeastRequests, PowerOfTwoChoices y FirstAlphabetical; el validador del ejercicio conserva su subconjunto original de cuatro. La política por defecto es `PowerOfTwoChoices`. La política se aplica a todas las peticiones del cluster. La política se documenta en la configuración.

```
{
  "Clusters": {
    "ordenes-cluster": {
      "LoadBalancingPolicy": "RoundRobin",
      "Destinations": {
        "ordenes-1": { "Address": "http://aceria-ordenes-1:5001/" },
        "ordenes-2": { "Address": "http://aceria-ordenes-2:5001/" }
      }
    }
  }
}
```

- **Línea 1:** `{` → abre un objeto en el contexto mostrado.
- **Línea 2:** `"Clusters": {` → abre la sección de clusters.
- **Línea 3:** `"ordenes-cluster": {` → declara el cluster.
- **Línea 4:** `"LoadBalancingPolicy": "RoundRobin",` → configura el algoritmo.
- **Línea 5:** `"Destinations": {` → abre la sección de destinos.

- **Línea 6:** `"ordenes-1": { "Address": "http://aceria-ordenes-1:5001/" },` → declara una dirección de ejemplo con nombre ASCII; debe resolverse en la red del emisor.
- **Línea 7:** `"ordenes-2": { "Address": "http://aceria-ordenes-2:5001/" }` → declara una dirección de ejemplo con nombre ASCII; debe resolverse en la red del emisor.
- **Línea 8:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 9:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 10:** `}` → cierra el objeto abierto en el contexto anterior.
- **Línea 11:** `}` → cierra el objeto abierto en el contexto anterior.

El algoritmo se caracteriza por tres propiedades. La primera es que se configura por cluster. La segunda es que determina la distribución. La tercera es que se documenta. Las tres propiedades permiten que el balanceo sea predecible.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. LoadBalancingPolicy selecciona una implementación registrada en el contenedor. La política se aplica sobre destinos disponibles, no necesariamente sobre toda la colección original. La selección predeterminada es PowerOfTwoChoices si no se indica otra.

Ejemplo razonado. La etapa 8.4 escribe RoundRobin para que la decisión sea visible y revisable. En 8.2, con un único destino, cambiar de algoritmo no añade redundancia ni produce un reparto observable entre procesos.

Riesgo, alternativa y límite. YARP incluye también FirstAlphabetical, útil para una selección preferente; el validador didáctico conserva su lista original de cuatro políticas. Esa lista local no pretende enumerar todas las extensiones posibles ni todos los valores admitidos por YARP.

Comprobación y aprendizaje. Localiza la opción efectiva del cluster y comprueba que el pipeline incluye balanceo. Un nombre correcto en JSON no garantiza el comportamiento si se sustituyó el pipeline estándar sin registrar el middleware necesario.

Bloque 6 — Balanceo y health checks

El balanceo se combina con los health checks para excluir las instancias no disponibles. YARP consulta periódicamente los health checks de cada destino. La política activa acumula los fallos y aplica su umbral antes de marcar un destino no disponible. Las peticiones se enrutan solo a los destinos disponibles. La combinación permite que el sistema se recupere de fallos de instancias. La integración se configura en el cluster.

```
Balanceo y health checks (HealthyAndUnknown):

Destino 1 → healthy → recibe tráfico.
Destino 2 → unhealthy → no recibe tráfico.
Destino 3 → healthy → recibe tráfico.
```

- **Línea 1:** `Balanceo y health checks (HealthyAndUnknown):` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Destino 1 → healthy → recibe tráfico.` → primer destino disponible.
- **Línea 4:** `Destino 2 → unhealthy → no recibe tráfico.` → segundo destino no disponible.
- **Línea 5:** `Destino 3 → healthy → recibe tráfico.` → tercer destino disponible.

La integración se caracteriza por tres propiedades. La primera es que los health checks excluyen las instancias no disponibles. La segunda es que el balanceo se ajusta de forma automática. La tercera es que el sistema se recupera de fallos de instancias. Las tres propiedades permiten que el balanceo sea resiliente.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Las sondas activas examinan readiness periódicamente y las políticas actualizan la salud por destino. Las pasivas usan resultados de tráfico. La detección tiene una demora y no puede impedir un fallo que ocurra después de haber seleccionado una instancia.

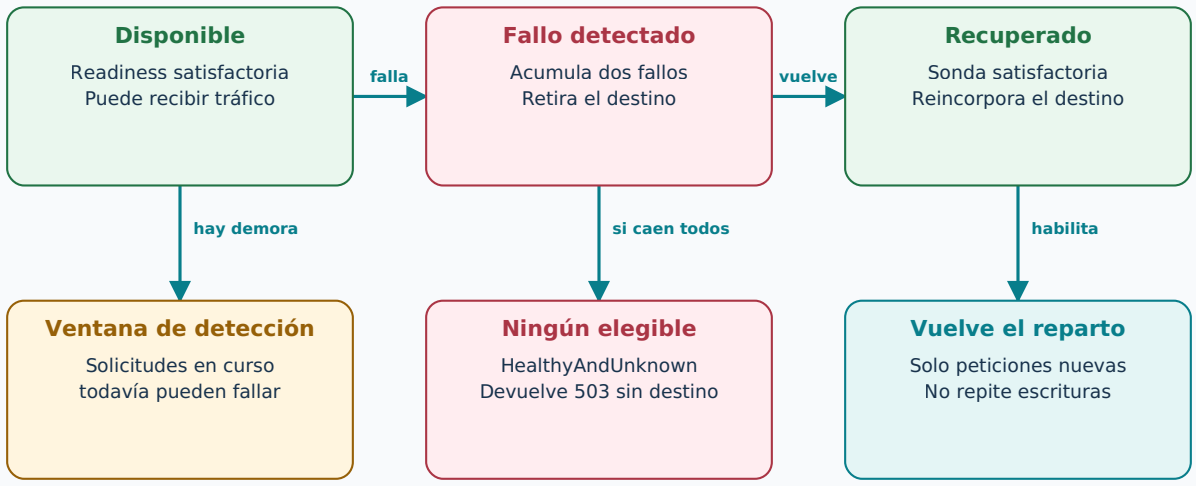
Ejemplo razonado. El ensayo final consulta /health/ready cada segundo, con timeout de un segundo y umbral de dos fallos consecutivos. HealthyAndUnknown excluye destinos unhealthy y produce 503 cuando no quedan candidatos; al recuperarse, una sonda satisfactoria permite reincorporarlos.

Riesgo, alternativa y límite. HealthyOrPanic es una alternativa que vuelve a habilitar todos si ninguno queda elegible. No expresa el mismo contrato que fallar sin backend sano. Los valores operativos se eligen para observar el laboratorio y no se presentan como ajuste universal de producción.

Comprobación y aprendizaje. Verifica tres escenarios separados: pierde una instancia, pierde ambas y recupera una. Espera a la condición observable con un límite, en vez de suponer una transición instantánea. No se reintentan automáticamente escrituras para ocultar una caída.

La salud cambia el conjunto de destinos elegibles

Sonda activa /health/ready: intervalo 1 s, timeout 1 s y umbral de 2 fallos.



HealthyOrPanic permite volver a intentar todos si ninguno queda sano. La etapa docente evita esa política de reserva.

JAIME GALLO · M08

Interpretación del gráfico: La fila superior expone estados operativos sucesivos y la inferior sus efectos. El umbral introduce una ventana durante la que una petición aún puede fallar al intentar el destino detenido. HealthyAndUnknown evita seleccionar servidores marcados unhealthy cuando todos han caído. Al observar recuperación se habilitan nuevas asignaciones; no se repite una escritura anterior ni se garantiza que una petición que ya estaba en curso termine bien.

Bloque 7 — Balanceo en contenedores

El balanceo en contenedores se realiza con el orquestador. Kubernetes incluye un servicio que balancea entre los pods de un deployment. Docker Swarm incluye un balanceo similar. El gateway también puede balancear entre las instancias de un servicio. La combinación del balanceo del orquestador y del gateway permite una distribución completa. La configuración se documenta por entorno.

```
Balanceo en contenedores:

Kubernetes → Service balancea entre pods.
Docker Swarm → Service balancea entre tasks.
Gateway → balancea entre instancias del servicio.
```

- **Línea 1:** `Balanceo en contenedores:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Kubernetes → Service balancea entre pods.` → asocia Kubernetes con Service balancea entre pods, según el alcance conceptual del ejemplo.
- **Línea 4:** `Docker Swarm → Service balancea entre tasks.` → asocia Docker Swarm con Service balancea entre tasks, según el alcance conceptual del ejemplo.
- **Línea 5:** `Gateway → balancea entre instancias del servicio.` → asocia Gateway con balancea entre instancias del servicio, según el alcance conceptual del ejemplo.

El balanceo en contenedores se caracteriza por tres propiedades. La primera es que el orquestador balancea entre instancias. La segunda es que el gateway enruta entre servicios y puede balancear entre instancias de cada servicio. La tercera es que ambos se complementan. Las tres propiedades permiten que el sistema sea escalable.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. La plataforma puede ofrecer una dirección de servicio que reparta entre réplicas, o YARP puede conocer cada destino. La elección determina quién descubre instancias y quién observa su salud. Duplicar capas no garantiza una distribución mejor.

Ejemplo razonado. El laboratorio local usa dos destinos explícitos y procesos controlados para observar identidad y fallos. SQL Server y RabbitMQ reales validan la continuidad del acumulado; esa infraestructura no convierte el ejercicio en un despliegue Kubernetes.

Riesgo, alternativa y límite. Si YARP apunta a una única dirección de Service, su health check observa ese acceso y no necesariamente cada pod. Una red de contenedores también cambia el significado de localhost. Deben documentarse el responsable del reparto y la resolución DNS.

Comprobación y aprendizaje. Representa la dirección que ve el gateway y enumera qué instancias puede observar directamente. Usa nombres ASCII en una red de contenedores y prueba desde el emisor real. No atribuyas al gateway conocimiento que solo tiene el orquestador.

Bloque 8 — Estrategias de escalado

El balanceo de carga se combina con estrategias de escalado. El escalado horizontal añade instancias cuando la carga aumenta. El escalado vertical aumenta los recursos de una instancia. El escalado automático ajusta el número de instancias según métricas. La combinación de balanceo y escalado permite que el sistema se adapte a la carga. La estrategia se documenta por servicio.

```
Estrategias de escalado:

Horizontal → añade instancias.
Vertical   → aumenta recursos.
Automático → ajusta instancias según métricas.
```

- **Línea 1:** `Estrategias de escalado:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Horizontal → añade instancias.` → asocia Horizontal con añade instancias, según el alcance conceptual del ejemplo.
- **Línea 4:** `Vertical → aumenta recursos.` → asocia Vertical con aumenta recursos, según el alcance conceptual del ejemplo.
- **Línea 5:** `Automático → ajusta instancias según métricas.` → asocia Automático con ajusta instancias según métricas, según el alcance conceptual del ejemplo.

Las estrategias se caracterizan por tres propiedades. La primera es que cada estrategia tiene un propósito. La segunda es que la elección depende de la carga. La tercera es que el balanceo distribuye entre las instancias. Las tres propiedades permiten que el sistema sea escalable.

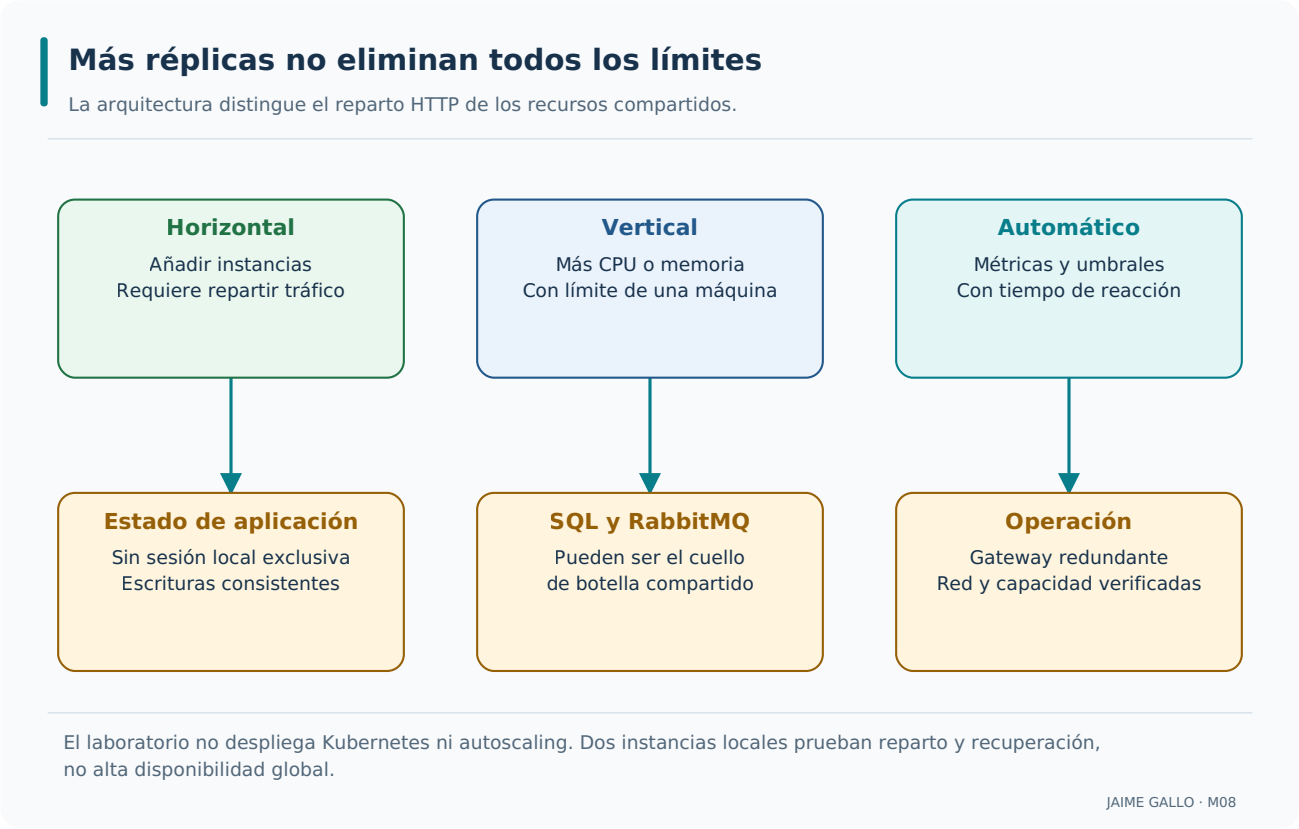
Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Escalar horizontalmente añade procesos; verticalmente aumenta recursos por proceso; automatizar requiere métricas, umbrales y una plataforma que ejecute cambios. El balanceador solo distribuye solicitudes entre los destinos que conoce.

Ejemplo razonado. Dos instancias de Inventario permiten ensayar reparto sin cambiar la API pública. Ambas acceden al estado persistente del servicio. El gateway, SQL y RabbitMQ requieren su propio análisis de capacidad y disponibilidad.

Riesgo, alternativa y límite. Añadir procesos puede aumentar la presión sobre SQL y agotar conexiones. Un autoscaler tampoco reacciona sin demora: necesita señales y tiempo de arranque. Más réplicas no corrigen bloqueos, consultas ineficientes ni efectos duplicados de una escritura.

Comprobación y aprendizaje. Observa latencia, errores y recursos antes de justificar otra instancia. Explica qué cuello de botella se pretende aliviar y cómo se comprobaría. El curso no instala autoscaling; conserva el concepto y delimita la evidencia obtenida con procesos locales.



Interpretación del gráfico: Las columnas separan tres formas de aumentar capacidad y sus condicionantes. El estado, las dependencias compartidas y la operación del punto de entrada siguen requiriendo diseño. Más procesos HTTP pueden trasladar la saturación a SQL. Los mecanismos de autoscaling y orquestación se estudian como alternativas; la evidencia del laboratorio corresponde a procesos locales y dependencias reales controladas, sin afirmar un despliegue Kubernetes.

Bloque 9 — Buenas prácticas de balanceo de carga

Las buenas prácticas de balanceo de carga son cinco. La primera es usar al menos dos instancias por servicio. La segunda es configurar el algoritmo adecuado al contexto. La tercera es combinar el balanceo con health checks. La cuarta es registrar las métricas de cada instancia. La quinta es validar la configuración con un test. La aplicación de las cinco prácticas produce un balanceo claro y resiliente.

```
Buenas prácticas de balanceo de carga:

1. Al menos dos instancias por servicio.
2. Algoritmo adecuado al contexto.
3. Combinar con health checks.
4. Registrar métricas por instancia.
5. Validar con un test.
```

- **Línea 1:** `Buenas prácticas de balanceo de carga:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `1. Al menos dos instancias por servicio.` → primera práctica.
- **Línea 4:** `2. Algoritmo adecuado al contexto.` → segunda práctica.
- **Línea 5:** `3. Combinar con health checks.` → tercera práctica.
- **Línea 6:** `4. Registrar métricas por instancia.` → cuarta práctica.

- **Línea 7:** `5. Validar con un test.` → quinta práctica.

Las buenas prácticas se caracterizan por tres propiedades. La primera es que producen un balanceo resiliente. La segunda es que permiten el diagnóstico. La tercera es que permiten el análisis automático. Las tres propiedades permiten que el balanceo sea un activo del sistema.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. Dos instancias son una condición útil para estudiar reparto, pero no garantizan alta disponibilidad. La salud de dependencias, la capacidad de la superviviente y la coherencia del estado condicionan el resultado del fallo.

Ejemplo razonado. La comprobación docente identifica cada instancia, conserva respuestas de negocio y observa retirada y recuperación. Los ensayos de catálogos se mantienen junto a las pruebas del proxy y las pruebas de integración con infraestructura real.

Riesgo, alternativa y límite. Una prueba que solo alterna un contador en memoria no demuestra persistencia compartida; una prueba que solo inspecciona dos URL no demuestra reparto. Deben conservarse cuerpos, códigos y efectos además de la identidad técnica.

Comprobación y aprendizaje. Recorre el estado inicial, la caída de una instancia, la caída total y la recuperación. Registra qué resultado distingue cada fase. Evita contar como éxito un test omitido o una respuesta sintética que no alcanzó ningún backend.

Bloque 10 — Aplicación al proyecto del curso y cierre del curso

El proyecto del curso aplica el balanceo de carga en el módulo 8. El cluster ejecutable de Inventario declara dos destinos; el de Producción conserva uno. La política de balanceo se configura en el cluster. Los health checks excluyen las instancias no disponibles. La validación del balanceo se realiza con un test que verifica la coherencia. El módulo 8 cierra con el resumen del estado del proyecto. El curso cierra con el resumen final del proyecto completo.

```
Aplicación al proyecto del curso:

Clusters          → Inventario: dos destinos; Producción: uno.
Política          → configurada por cluster.
Health checks     → excluyen instancias no disponibles.
Validación        → test de coherencia.
Cierre del módulo → resumen del estado del proyecto.
Cierre del curso  → resumen final del proyecto completo.
```

- **Línea 1:** `Aplicación al proyecto del curso:` → introduce el conjunto de conceptos que se desarrolla en las líneas siguientes.
- **Línea 2:** (línea vacía) → separa visualmente dos partes del ejemplo sin ejecutar una operación.
- **Línea 3:** `Clusters → Inventario: dos destinos; Producción: uno.` → describe la topología ejecutable final: dos destinos de Inventario y uno de Producción.
- **Línea 4:** `Política → configurada por cluster.` → asocia Política con configurada por cluster, según el alcance conceptual del ejemplo.
- **Línea 5:** `Health checks → excluyen instancias no disponibles.` → asocia Health checks con excluyen instancias no disponibles, según el alcance conceptual del ejemplo.
- **Línea 6:** `Validación → test de coherencia.` → asocia Validación con test de coherencia, según el alcance conceptual del ejemplo.
- **Línea 7:** `Cierre del módulo → resumen del estado del proyecto.` → asocia Cierre del módulo con resumen del estado del proyecto, según el alcance conceptual del ejemplo.

- **Línea 8:** `Cierre del curso → resumen final del proyecto completo.` → asocia Cierre del curso con resumen final del proyecto completo, según el alcance conceptual del ejemplo.

El cierre del curso tiene dos consecuencias prácticas. La primera es que el proyecto reúne los mecanismos docentes de arquitectura de microservicios estudiados, con sus límites operativos explícitos. La segunda es que el alumno dispone de todos los artefactos acumulados. La combinación de ambas permite que el alumno aplique lo aprendido a proyectos reales.

Ampliación didáctica aplicada a AceriaApp

Decisión y fundamento. El cierre reúne la arquitectura acumulativa: APIs, límites hexagonales, patrones, DDD/CQRS, HTTP y mensajería, observabilidad, resiliencia y gateway. La coherencia del recorrido se demuestra con artefactos y resultados reproducibles.

Ejemplo razonado. ClusterBalanceo conserva el catálogo de tres clusters y siete destinos declarados. Su reto produce total 3, destinos 7, clusters con pesos 1 y con health checks 3. El proxy real usa dos servicios, con dos instancias de Inventario y RoundRobin.

Riesgo, alternativa y límite. Los 307 tests de la conversación original no representan el repositorio actual: M07 aporta 403 pruebas y M08 conserva 42 de catálogo más las funcionales nuevas. Las cifras finales se publican solo a partir de ejecuciones verificadas, manteniendo los retos aislados.

Comprobación y aprendizaje. El alumno debe poder reconstruir cada etapa desde las prácticas, ejecutar los ensayos y explicar sus límites. Ese resultado permite trasladar criterios a otro proyecto; no certifica un despliegue de producción ni una disponibilidad global que el laboratorio no ha medido.

Resumen rápido de la teoría

- El balanceo de carga distribuye el tráfico entre varias instancias de un servicio.
- Los algoritmos estudiados son round-robin, random, least-requests, power-of-two-choices y ponderación; esta última necesita una política personalizada en YARP.
- La configuración de destinos múltiples se realiza en la sección `Destinations` del cluster.
- Los pesos de destino permiten distribuir la carga de forma ponderada.
- YARP soporta varios algoritmos con la propiedad `LoadBalancingPolicy`.
- El balanceo se combina con los health checks para excluir instancias no disponibles.
- El balanceo en contenedores se realiza con el orquestador.
- Las estrategias de escalado son horizontal, vertical y automático.
- Las buenas prácticas son al menos dos instancias, algoritmo adecuado, combinar con health checks, registrar métricas y validar con test.
- El módulo 8 cierra con el resumen del estado del proyecto. El curso cierra con el resumen final.

Fuentes técnicas y criterios de lectura

La documentación oficial respalda los contratos de YARP utilizados; los fragmentos del curso se adaptan al laboratorio y la versión NuGet fijada. Los enlaces de Microsoft Learn pueden mostrar la edición vigente del sitio. Para reproducir el proyecto se mantienen net8.0 y Yarp.ReverseProxy 2.2.0.

- Configuración de rutas, clusters y destinos de YARP: jerarquía ReverseProxy, enlace con IConfiguration y ejemplos de opciones.
- Transformaciones de petición: PathRemovePrefix, RequestHeader, RequestHeaderRemove, QueryValueParameter y QueryRouteParameter; aplicación en el orden configurado.
- Transformaciones de petición y respuesta y extensibilidad: transformaciones en código y configuración de rutas.

- Balanceo en YARP: políticas incorporadas y extensión mediante ILoadBalancingPolicy.
- DestinationConfig de YARP 2.2: propiedades Address, Health, Host y Metadata; no existe Weight incorporada.
- Salud de destinos: salud activa y pasiva, ConsecutiveFailures y políticas HealthyAndUnknown y HealthyOrPanic.
- Coincidencia por cabeceras y precedencia: condiciones de Match y efecto de Order.

Cierre del módulo y del curso

El recorrido conserva los ocho módulos: introducción a microservicios, APIs REST, arquitectura hexagonal, patrones, DDD/CQRS, comunicación, observabilidad/resiliencia y publicación con gateway. En M08 las declaraciones de catálogo se contrastan con un proceso de proxy independiente, rutas publicadas, adaptación de mensajes y reparto entre instancias. El alumno puede explicar qué servicio decide cada resultado y qué prueba sostiene cada afirmación.

El material práctico conserva cuatro etapas acumulativas, cuatro retos aislados y los documentos de decisión correspondientes. Reconstrucción, pruebas de comportamiento y evidencia de integración se verifican sobre archivos concretos. La persistencia SQL y la mensajería mantienen los contratos anteriores; el gateway aporta un borde HTTP y no sustituye esas responsabilidades. El cierre de una ejecución requiere los informes efectivos del proyecto y no se deduce del número histórico de ejemplos del temario.